

claude-windows / 96963dab-d38f-488c-af44-530ddbfeed1

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbfeed1\subagents\workflows\wf\_00bebd3c-82f\agent-a7e28066c5474c821.jsonl`
- SHA-256: `60f218fd6a298c78bc5bd40734e9bbbed2e38108e64e1897a321a6a84b2d681a1`
- Source modified: `2026-07-08T05:50:53+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf\_00bebd3c-82f`
- session_id: `96963dab-d38f-488c-af44-530ddbfeed1`

Transcript

1. user (2026-07-08T05:44:49.640Z)

You are mapping a subsystem of the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (Windows checkout; read-only ? do NOT edit anything).
Context: GitHub issue #524 wants to (a) move validation onto the L4 Cloud Run Job worker via a config-driven phase_placement knob, (b) drop a redundant re-hash in /api/process by trusting the upload's streamed video_id, and (c) design a topology where upload->stitch runs on the L4 with async copy-to-bucket. You are producing the precise code map a designer/implementer needs. Cite exact file paths and line numbers. Read the actual code, not just docs. Follow imports where the flow crosses files.
Subsystem: CONFIG SYSTEM. Map backend/config/ (Pydantic models): the deployment section (compute_target etc.), every knob that decides WHERE a phase runs today, how config reaches the Cloud Run worker (env vars? args? baked into image?), config file layout (yaml? env?), and how deploy/gcp/ sets these for the live control-plane + rally-worker job. Issue #521 plans a per-phase placement knob (validation/detect/compile -> control_plane|cloud_run_l4). Propose (from the existing conventions) the exact shape and location such a knob would take so a new field slots in idiomatically ? cite the closest existing precedent field and its plumbing.

2. user (2026-07-08T05:44:54.587Z)

```
backend\config\__init__.py
backend\config\loader.py
backend\config\startup.py
backend\config\models.py
backend\config\presets.py
backend\config\__pycache__\__init__.cpython-313.pyc
backend\config\__pycache__\loader.cpython-313.pyc
backend\config\__pycache__\models.cpython-313.pyc
backend\config\__pycache__\presets.cpython-313.pyc
backend\config\__pycache__\startup.cpython-313.pyc
```

3. user (2026-07-08T05:44:55.507Z)

```
deploy\gcp\nightly_regression\Dockerfile.launcher
deploy\gcp\nightly_regression\cloudbuild.launcher.yaml
deploy\gcp\nightly_regression\run_on_vm.sh
deploy\gcp\ops-agent-startup.sh
deploy\gcp\buckets.env
deploy\gcp\create_gpu_vm.sh
deploy\gcp\nightly_regression\create_nightly_bucket.sh
deploy\gcp\nightly_regression\README.md
deploy\gcp\README.md
deploy\gcp\nightly_regression\launch.sh
deploy\gcp\nightly_regression\register_scheduler.sh
deploy\gcp\nightly_regression\setup_email.sh
deploy\gcp\provision.sh
```

deploy\gcp\teardown.sh

4. user (2026-07-08T05:45:03.639Z)

```
1      """Pydantic models for the application configuration.
2
3      This module defines the canonical, validated, typed configuration for the
4      Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6      Unknown / typo'd config keys are rejected at load time via extra="forbid".
7      """
8
9      from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         pts_max_gap_seconds: float = Field(
31             default=10.0,
32             ge=0.0,
33             description=(
34                 "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
35                 "GoPro H.264/H.265 recordings typically have GOP intervals of 1?4s; "
36                 "a gap >10s suggests edited/missing content. Mobile phones, screen "
37                 "recordings, and other encoders may use different GOP sizes ? tune "
38                 "this threshold for your recording device."
39             ),
40         )
41         relevance: ValidationRelevanceConfig = Field(
42             default_factory=ValidationRelevanceConfig
43         )
44
45
46     class TrackNetConfig(BaseModel):
47         wsl_distro: str = "Ubuntu"
48         repo_dir: str = "~/models/TrackNetV4"
49         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50         conda_env: str = "TrackNetV4"
51         weights_path: str = ""
52         queue_length: int = 5
53         stage_in_wsl: bool = True
54         wsl_stage_dir: str = "~/clips"
55         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
56         velocity_threshold: float = 0.02
57         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58         # decoded frame cache) are deleted automatically ? they are pure intermediates,
59         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61         keep_frames: bool = False
```

```

62         # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
63         # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
64         # 0 = no extra floor (the headroom guard still applies).
65         min_free_gb: float = 0.0
66
67
68     class WasbIndexConfig(BaseModel):
69         """Typed config for the live WASB shuttle substrate (indexing.wasb).
70
71         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
72         these knobs actually take effect ? previously this whole block was silently dropped
73         because nested config sections defaulted to extra='ignore'.
74         """
75
76         wsl_distro: str = "Ubuntu"
77         repo_dir: str = "~/models/WASB-SBDT"
78         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
79         conda_env: str = "wasb"
80         weights_path: str = (
81             "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
82         )
83         sport: str = "badminton"
84         wsl_stage_dir: str = "~/clips"
85         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86         velocity_threshold: float = 0.02
87         # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
---
88         # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
89         # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
90         # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
91         device: str = "cuda" # auto | cuda | mps | cpu
92         python_bin: str = (
93             "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
94         )
95         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
96         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
97         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
98         keep_frames: bool = False
99         # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
100        # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
101        # 0 = no extra floor (the headroom guard still applies).
102        min_free_gb: float = 0.0
103        # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
104        # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
105        # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
106        # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
107        # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
108        # Tri-state:
109        #     None          = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
110        #                     behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
111        #     True/False = force on/off for BOTH runners (e.g. set False for a container cv2

```

```

can't seek).
112     stream_video: Optional[bool] = None
113     # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
114     # 2026-07-03: the default batch 8 with single-process loading left the GPU <1%
utilized ?
115     # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8,
no
116     # prefetch ? byte-identical to the historical path). The cloud-serving preset opts
into
117     # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
118     batch_size: Optional[int] = None
119     # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
120     # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
121     prefetch_batches: Optional[int] = None
122     # GPU-RESIDENT DECODE (#506) ? DEFAULT OFF. "nvdec_resident" moves
decode+resize+normalize
123     # onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
`grid_sample`
124     # resize replicating WASB's cv2.warpAffine EXACTLY (the #506 lesson: a naive
F.interpolate
125     # silently over-detects ? F1 0.58?0.07 ? and only shows on hard clips) ? GPU
ImageNet
126     # normalize ? HRNet. Validated on a real L4 (GX010128, SERVED preset, 2026-07-06):
~1.7x
127     # fps (35.8?60.2), GPU util 47?79% / CPU 62?35%, F1 0.574 ? baseline 0.582. Native
(Linux)
128     # runner only ? needs the torch-2.x + PyNvVideoCodec WASB env
(deploy/cloudrun/Dockerfile)
129     # with libnvcuvid/libnvidia-encode present, and device=cuda; the WSL runner ignores
it.
130     # Flip gate: the full 15-clip golden set at the SERVED preset (GX010128 must pass).
131     decode_backend: Literal["cpu", "nvdec_resident"] = "cpu"
132
133     @model_validator(mode="after")
134     def _nvdec_resident_requires_cuda(self) -> "WasbIndexConfig":
135         # NVDEC decode is CUDA-only; reject the contradictory combo at config load
instead of
136         # failing deep inside a paid GPU run. "auto" is allowed here ? the native
runner's
137         # healthcheck hard-fails if auto resolves to cpu with this backend on.
138         if self.decode_backend == "nvdec_resident" and self.device not in ("cuda",
"auto"):
139             raise ValueError(
140                 f"indexing.wasb.decode_backend='nvdec_resident' requires device 'cuda'
or "
141                 f"'auto' (got {self.device!r}) ? NVDEC decode runs on the GPU."
142             )
143         return self
144
145     @field_validator("batch_size")
146     @classmethod
147     def _batch_size_positive(cls, v: Optional[int]) -> Optional[int]:
148         # A DataLoader batch must be >=1; reject at config time so a typo can't reach
149         # `wasb_infer.py --batch-size` as an invalid value (the runner only passes the
flag
150         # when truthy, so 0 stays "off/parity-default").
151         if v is not None and v < 1:
152             raise ValueError(f"indexing.wasb.batch_size must be >= 1 when set (got
{v})")
153         return v
154
155     @field_validator("prefetch_batches")
156     @classmethod
157     def _prefetch_batches_non_negative(cls, v: Optional[int]) -> Optional[int]:
158         # 0 = off (the default). Reject negatives: the runner passes any truthy value as

```

```

159         # '--prefetch-batches', and `_PrefetchIter` clamps depth with max(1,
int(depth)), so a
160         # negative would SILENTLY enable prefetch at depth 1 instead of failing loudly.
161         if v is not None and v < 0:
162             raise ValueError(
163                 f"indexing.wasb.prefetch_batches must be >= 0 when set (got {v})"
164             )
165         return v
166
167
168     class FusionConfig(BaseModel):
169         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
170
171         Every default reproduces control behaviour: the fusion segmenter persists the
172         shuttle net-crossing count (Gate A signal) and ? only when `cue_flags['person']`
is
173         true ? the person-cue features, but it does NOT gate the served handoff unless a
174         threshold is raised (`min_crossings`/`min_players` default 0 = OFF). The court
mask
175         is optional: `court_polygon=None` ? Gate B counts every detection (player-count-
only);
176         supplied ? off-court persons (spectators / adjacent courts) are excluded.
177         """
178
179         # Which trajectory substrate seeds the windows (shuttle stays primary).
180         substrate: Literal["wasb", "tracknet"] = "wasb"
181         # Windowing velocity threshold for the fusion family (the engine reads
182         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
183         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
184         velocity_threshold: float = 0.02
185         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
186         # enabled ? so the default path never imports torch / touches the GPU.
187         cue_flags: dict[str, bool] = Field(default_factory=dict)
188         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
189         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
190         min_crossings: int = Field(default=0, ge=0)
191         # Served Gate B: when the person cue is on, drop windows with median in-court
players
192         # < this. 0 = OFF.
193         min_players: int = Field(default=0, ge=0)
194         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
195         court_polygon: Optional[list[list[float]]] = None
196         # Net line for the person two-sided-motion split (person features only ? the shuttle
197         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
198         net_axis: Literal["x", "y"] = "y"
199         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
200
201
202     class IndexingConfig(BaseModel):
203         default_segmenter: str = "yolo_hybrid"
204         use_gpu: bool = True
205         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
206         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
207         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
208         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
209         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
210         detector_impl: str = "auto"
211         motion_threshold: float = 0.08
212         min_segment_duration: float = 3.0
213         dead_time_merge_gap: float = 2.0
214         chunk_duration: float = 90.0

```

```

215     chunk_overlap: float = 10.0
216     detector_model: str = "fasterrcnn_resnet50_fpn"
217     detector_score_threshold: float = 0.5
218     yolo_velocity_threshold: float = 0.15
219     yolo_frame_skip: int = 3
220     yolo_tracker: str = "bytetrack.yaml"
221     yolo_prefetch_workers: int = 2
222     min_action_window_duration: float = 2.0
223     # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
224     # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
225     # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
226     # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
227     # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
228     # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
229     # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
230     max_window_duration: float = 6.0
231     window_min_split_gap: float = (
232         0.5 # min internal gap (s) that qualifies as a split point
233     )
234     # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
235     # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
236     # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
237     # Only cuts (recall can't drop). OFF by default until measured/promoted.
238     window_hard_cap: bool = False
239     # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
240     # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
241     # [?end] gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
242     # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
243     # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
244     # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, [?end] 4.96?3.31s (n=13).
The W1 cap
245     # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
246     window_inactivity_end: float = 1.5
247     inactivity_rally_thresh: float = 0.20
248     inactivity_min_density: float = 0.30
249     # R4 density DENOMINATOR fix (code-review finding B). The R4 trim's fast-fraction
denominator is
250     # the COUNT of active samples in the trailing window, which sparse tracking inflates
(one fast
251     # sample after a gap ? 100% dense ? tail held open). When True, the denominator is
the window's
252     # TEMPORAL length instead. No-op under dense tracking (sample-count == temporal-
span), so it only
253     # corrects the sparse-track case. *** P2b PROMOTED DEFAULT-ON (2026-06-29): golden-
set A/B at the
254     # SERVED operating point (n=15, 595 rallies) ? tolF1 0.353?0.364 (?tolF1 +0.010,
sign 10W/4L, NO
255     # regression), [?end] 2.97?2.72s, seg_count_ratio ?0.103 (p=0.006), split_rate
?0.010 (p=0.016),

```

```

256         # R1 over-seg guard PASS; served_regresses(eps=0.0)=False ?
promote_if_served_safe(structural)=True.
257         # See docs/RALLY_DETECTION_FIXES_PLAN.md §7/§8. ***
258         inactivity_temporal_density: bool = True
259         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
260         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
261         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
262         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
263         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
264         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
265         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
266         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
267         window_blob_fallback: bool = False
268         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
269         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
270         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
271         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
272         window_blob_factor: float = 4.0
273         log_yolo_candidates: bool = True
274         store_yolo_candidates: bool = True
275         ai_handoff_padding: float = 2.0
276         ai_max_workers: int = 5
277         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
278         # chunks of a high-bitrate (4K) source blow past the provider upload cap
279         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
280         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
281         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
282         # Matches gemini_refine's --clip-scale-width default.
283         ai_chunk_scale_width: int = 1280
284         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
285         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
286         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
287         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
288         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
289         skip_ai_handoff: bool = False
290         # Optional debug knob: trim every video to the first N seconds before processing.
291         debug_duration: Optional[float] = None
292
293         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
294         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
295         fusion: FusionConfig = Field(default_factory=FusionConfig)
296
297         @field_validator("default_segmenter")
298         @classmethod
299         def segmenter_must_be_registered(cls, v: str) -> str:
300             # Lazy import: the segmenter registry imports modules that may import
301             # config, so we resolve it at validation time rather than at module load.
302             from backend.pipeline.segmenters.base import segmenter_registry
303
304             if segmenter_registry.get(v) is None:

```

```

305         available = sorted(segmenter_registry.list_available().keys())
306         raise ValueError(
307             f"Segmenter '{v}' is not registered. Available: {'', '.join(available)}"
308         )
309     return v
310
311     @model_validator(mode="after")
312     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
313         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
314         # step makes `while t < duration: t += step` loop forever, growing memory before
any
315         # GPU work. Reject the bad config at load time instead.
316         if self.chunk_overlap >= self.chunk_duration:
317             raise ValueError(
318                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
319                 f"({self.chunk_duration}); otherwise chunking never advances."
320             )
321         return self
322
323
324     class OutputConfig(BaseModel):
325         default_highlights_name: str = "highlights.mp4"
326         db_name: str = "sports_indexer.db"
327         # Directory where the DB, highlight reels, and processed clips are written.
328         # The CLI's --output-dir overrides this at startup (see backend/main.py:main).
329         output_dir: str = "output"
330
331
332     class WatermarkConfig(BaseModel):
333         """Branding overlay burned into each highlight clip during the per-clip re-encode
334         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
335         (under `stitching.watermark` in config.json) to turn it off. The text is fully
336         configurable. The concat stage stays stream-copy (-c copy)."""
337
338         enabled: bool = True
339         text: str = "Generated by Khelsutra.guru"
340         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
341         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family below
342         font: str = "Sans" # fontconfig family used when font_path is empty
343         font_size_ratio: float = Field(
344             default=0.035, gt=0.0, le=0.5
345         ) # text height as a fraction of frame height
346         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
347         box: bool = True # faint backing box behind the text for legibility
348         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
349         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = (
350             "bottom-right"
351         )
352
353
354     class StitchingConfig(BaseModel):
355         begin_padding: float = 0.5
356         end_padding: float = 0.5
357         use_cache: bool = False
358         min_shot_count: int = 1
359         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
360         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
361         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
362         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
363         # local detector over-fires and its false positives are mostly SHORT (motion blips,

```



```

364         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
365         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
366         min_rally_duration: float = Field(default=0.0, ge=0.0)
367         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
368
369
370     class LoggingConfig(BaseModel):
371         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
372
373         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
374         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
375         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
376         # them to WARNING; set true to restore full chatter for request-flow debugging.
377         verbose_http: bool = False
378
379
380     class SecurityConfig(BaseModel):
381         # When set, /api/process video_path must resolve under this directory (hosted/tenant
382         # mode). Default None preserves the single-user local 'any local file' workflow.
383         allowed_video_root: Optional[str] = None
384
385         # --- Chunked upload (B2, PR-2) ---
386         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
387         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
388         # contain upload_dir or /api/process will reject the uploaded paths.
389         upload_dir: str = "output/uploads"
390         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
391         # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
392         max_upload_mb: int = 4096
393         max_part_mb: int = 95
394         allowed_upload_extensions: List[str] = ["mp4", "mov"]
395
396         # --- M9 edge auth (Cloudflare Access, D9) ---
397         # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
398         # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
399         # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
400         # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
401         # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
402         # application's audience tag. NO secrets here ? these are public identifiers.
403         edge_auth_enabled: bool = False
404         cf_team_domain: str = (
405             "" # e.g. https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
406         )
407         cf_access_aud: str = "" # the CF Access application AUD tag the token must carry
408         # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
409         # backend/main.py so importing the app does no filesystem I/O; default ["*"].
Credentials stay
410         # OFF ? Cf-Access is a header, not a cookie.)
411
412
413     class StorageConfig(BaseModel):
414         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + D0
Spaces
415
416         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
417         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to

```

```

the constructor ?
418     **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
419     (boto3/google auth chains supply credentials). See ``backend/storage/``.
420
421     backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
422     # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
423     output_root: str = ""
424     # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
425     # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
426     # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
427     # input name is resolved against ``input_root`` (e.g.
``input_root``="gs://bucket/videos"`` + "x.mp4"
428     # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
429     # (``"gcs"`` + "bucket/x.mp4" => ``gs://bucket/x.mp4``). An explicit scheme on the
input
430     # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
431     # scratch dir before processing (see ``backend.storage.acquire_local_input``).
432     input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
433     input_root: str = ""
434     # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
435     # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
436     # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
437     # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
438     # the legacy flat layout is used unchanged.
439     single_folder_per_video: bool = False
440     # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
441     workspace_root: str = ""
442     # Cloud namespace prefix under the root so per-video folders sit tidily beside
443     # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
444     workspace_prefix: str = "workspaces"
445     local: dict[str, Any] = Field(default_factory=dict)
446     s3: dict[str, Any] = Field(default_factory=dict)
447     gcs: dict[str, Any] = Field(default_factory=dict)
448     gdrive: dict[str, Any] = Field(default_factory=dict)
449     # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
450     # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
451     gdrive_api: dict[str, Any] = Field(default_factory=dict)
452
453
454     class DeploymentConfig(BaseModel):
455         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
456         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
457
458         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
459         behaves exactly as before this section existed. A profile is opt-in (``config.json``
``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
460         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
461         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING

```

```

knobs
463         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
464
465         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
466
467         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
468         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
469         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
470         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
471         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
472         # Cloud-serving throughput cap (#466): how many DETECT jobs the control-plane drain
may dispatch +
473         # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound
Cloud Run Job
474         # execution (the GPU work is on ephemeral L4s, WASB-only has no Gemini), so ? unlike
a truly-local
475         # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
476         # ignore this and stay serial (max_workers=1), preserving the truly-local GPU/Gemini
serialization.
477         max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
478         # Poll budget for one dispatched remote job (CONTROL_PLANE_DURABLE_REBUILD $4.3).
Must sit
479         # ABOVE the worker Job's own --task-timeout (3600s) so the worker's timeout stays
the
480         # authority and the dispatcher's one-shot post-timeout marker rescue still applies ?
the old
481         # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect +
queue/cold-start,
482         # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid
GPU run.
483         remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
484
485
486     class BillingConfig(BaseModel):
487         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
488
489         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
490         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
491         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
492         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
493         inserts a real, calibrated version and points ``pricebook_version`` at it.
494
495         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
496         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
497         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
498
499         enabled: bool = False
500         pricebook_version: str = (
501             "placeholder-v0" # the seeded $0 version; owner flips to a real one
502         )
503         fx_inr_per_usd: float = 83.0 # USD?INR display rate (Bangalore market)
504         margin: float = 0.0 # resale markup (?0); 0 = bill at cost
505
506
507     class FlywheelConfig(BaseModel):

```

```

508         """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
509         loop: on a CONSENTED adult job, durably **capture** the source + rally output into
the per-video
510         workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden
TRAINING set, so
511         the owned model climbs toward the D11 ship floor.
512
513         **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-
identical to
514         today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even
``enabled=True``
515         captures **nothing**, because ``flywheel.consent_attested`` currently returns
``False`` for every
516         job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-
delete-raw).
517         This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent
columns ?
518         wiring) + the owned-model + golden-label data needed to *activate* shadow-
eval/promotion are
519         owner/counsel/GPU scope. Pairs with M9-consent."""
520
521         enabled: bool = False
522         workspace_prefix: str = (
523             "flywheel" # per-video workspace prefix for captured artifacts
524         )
525         shadow_eval: bool = True # run the owned-vs-Gemini shadow comparison on capture
526         min_promote_n: int = 5 # promote_if_served_safe `min_n` (the served-safe gate)
527
528
529     class AppConfig(BaseModel):
530         """Root configuration object.
531
532         All runtime configuration should be accessed via an instance of this class
533         (or the loader) rather than raw dict lookups.
534         """
535
536         sport: Sport = "badminton"
537         ai_provider: str = "gemini"
538         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
539         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
540         ai_model: str = "gemini-flash-latest"
541
542         validation: ValidationConfig = Field(default_factory=ValidationConfig)
543         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
544         output: OutputConfig = Field(default_factory=OutputConfig)
545         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
546         security: SecurityConfig = Field(default_factory=SecurityConfig)
547         logging: LoggingConfig = Field(default_factory=LoggingConfig)
548         storage: StorageConfig = Field(default_factory=StorageConfig)
549         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
550         billing: BillingConfig = Field(default_factory=BillingConfig)
551         flywheel: FlywheelConfig = Field(default_factory=FlywheelConfig)
552
553         # Reject unknown keys ? typed config migration is complete (D3).
554         model_config = {
555             "extra": "forbid",
556             "validate_assignment": True,
557         }
558
559
560     # Convenience type alias for callers that want to annotate
561     Config = AppConfig

```

5. user (2026-07-08T05:45:08.881Z)

```

1      """Configuration loader and saver.
2
3      Provides a single place to load validated AppConfig with built-in defaults
4      merged with optional file overrides (config.json).
5
6      This replaces scattered json.load + .get() patterns throughout the codebase.
7      """
8
9      from __future__ import annotations
10
11     import json
12     import logging
13     import os
14     from pathlib import Path
15     from typing import Any
16
17     from pydantic import BaseModel
18
19     from backend.utils.app_home import default_config_path
20
21     from .models import AppConfig
22     from .presets import (
23         COMPUTE_TARGET_FOR_PROFILE,
24         PROFILE_PRESETS,
25         is_known_profile,
26         preset_for,
27         suggest_profile,
28     )
29
30     logger = logging.getLogger(__name__)
31
32     # Config path resolution now lives in ``backend.utils.app_home.default_config_path()`` ?
33     it honours
34     # ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a) and equals ``Path("config.json")`` (CWD-
35     relative) when
36     # the env is unset, exactly the pre-P0a value. (The old module-level
37     ``DEFAULT_CONFIG_PATH`` constant
38     # was dropped: it had no external consumers and a frozen-at-import snapshot would
39     silently diverge
40     # from a later RALLY_APP_HOME.)
41
42
43     def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
44         """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
45         scalars
46         replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
47         overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
48         profile
49         path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
50         out = dict(base)
51         for key, val in override.items():
52             if isinstance(val, dict) and isinstance(out.get(key), dict):
53                 out[key] = _deep_merge(out[key], val)
54             else:
55                 out[key] = val
56         return out
57
58
59     def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
60         """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if
61         none

```

```

55         is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
a bare
56         environment never silently selects a profile (and never silently spends on cloud).
57
58         An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
file's
59         profile (rather than suppressing it) ? so an env typo can't leave the recorded
profile
60         asserting a preset that was never applied. A bad value in config.json is left to
surface as
61         a hard, typed-Literal error downstream (config-file correctness)."""
62         env_profile = os.environ.get("DEPLOYMENT_PROFILE")
63         if env_profile and env_profile.strip():
64             ep = env_profile.strip()
65             if is_known_profile(ep):
66                 return ep
67             logger.warning(
68                 "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
it "
69                 "and falling back to config.json deployment.profile.",
70                 ep,
71                 ", ".join(PROFILE_PRESETS),
72             )
73             # fall through to the file's profile (env typo must not suppress a valid file
choice)
74         dep = file_data.get("deployment")
75         if isinstance(dep, dict):
76             prof = dep.get("profile")
77             if isinstance(prof, str) and prof.strip():
78                 return prof.strip()
79         return ""
80
81
82     def _unknown_key_paths(
83         data: dict[str, Any], model_cls: type[BaseModel], prefix: str = ""
84     ) -> list[str]:
85         """Dotted paths in `data` that the typed config will not honor.
86
87         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
88         kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
89         typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
90         user's intent is lost ? collect every such key so load_config can warn.
91         """
92         unknown: list[str] = []
93         fields = model_cls.model_fields
94         for key, val in data.items():
95             if key not in fields:
96                 unknown.append(prefix + key)
97                 continue
98             ann = fields[key].annotation
99             if (
100                 isinstance(val, dict)
101                 and isinstance(ann, type)
102                 and issubclass(ann, BaseModel)
103             ):
104                 unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}.")
105         return unknown
106
107
108     # Cached model defaults ? Pydantic field defaults are class-level constants
109     # that never change at runtime. Initialised lazily on first call to
110     # _get_builtin_defaults() so importing this module is still cheap.
111     _BUILTIN_DEFAULTS_CACHE: dict[str, Any] | None = None
112
113

```

```

114 def _get_builtin_defaults() -> dict[str, Any]:
115     """Return a plain dict of the current built-in defaults.
116
117     The AppConfig model is the single source of truth for defaults.
118     Result is cached ? Pydantic field defaults never change at runtime.
119     (The live config returned by load_config() is NOT cached ? it is
120     a fresh AppConfig instance every call, reflecting config.json/env.)
121     """
122     global _BUILTIN_DEFAULTS_CACHE
123     if _BUILTIN_DEFAULTS_CACHE is None:
124         _BUILTIN_DEFAULTS_CACHE = AppConfig().model_dump(mode="json")
125     return _BUILTIN_DEFAULTS_CACHE
126
127
128 def load_config(path: str | Path | None = None) -> AppConfig:
129     """Load configuration with the following precedence:
130
131     1. Built-in defaults defined in the Pydantic models.
132     2. Values from the JSON file (if present and readable).
133     3. (Future) environment / CLI overrides.
134
135     Missing file or unreadable file ? returns a fully defaulted AppConfig.
136     Unknown keys in the file are tolerated (extra="allow" on the model).
137     """
138     cfg_path = Path(path) if path is not None else default_config_path()
139
140     file_data: dict[str, Any] = {}
141     if cfg_path.exists():
142         try:
143             with open(cfg_path, "r", encoding="utf-8") as f:
144                 file_data = json.load(f)
145         except Exception as exc:
146             # Non-fatal: continue with defaults + whatever we could parse.
147             # In production we might log here.
148             logger.warning(
149                 f"Failed to read {cfg_path}: {exc}. Using defaults + partial data."
150             )
151
152     # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`,
153     # `null`) would
154     # otherwise crash startup: a truthy non-mapping hits `.items()` in
155     # _unknown_key_paths, and any
156     # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
157     warn, per the
158     # loader's "bad input is non-fatal" contract.
159     if not isinstance(file_data, dict):
160         logger.warning(
161             "%s is not a JSON object (got %s); ignoring it and using defaults.",
162             cfg_path,
163             type(file_data).__name__,
164         )
165         file_data = {}
166
167     if file_data:
168         unknown = _unknown_key_paths(file_data, AppConfig)
169         if unknown:
170             logger.warning(
171                 "[CONFIG WARNING] %s contains keys the typed config does not "
172                 "recognize (typo'd or removed knobs ? top-level extras are kept "
173                 "but unread; unknown section keys are silently dropped): %s",
174                 cfg_path,
175                 ", ".join(sorted(unknown)),
176             )
177
178     # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied

```

```

at
176         # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
177         # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
With no
178         # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
179         defaults = _get_builtin_defaults()
180         profile = _resolve_explicit_profile(file_data)
181
182         if profile and not is_known_profile(profile):
183             # Only an unrecognised value from config.json reaches here (env typos already
fell back
184             # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
the
185             # typed Literal at model_validate below, a hard clear error ? loud, never
silent.
186             logger.warning(
187                 "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
188                 profile,
189                 ", ".join(PROFILE_PRESETS),
190             )
191             profile = ""
192
193         if profile:
194             merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
195             # Record the profile actually applied ? the env may have chosen it over
config.json,
196             # so re-stamp it onto the merged deployment section to keep the record honest.
197             merged.setdefault("deployment", {})
198             if isinstance(merged["deployment"], dict):
199                 merged["deployment"]["profile"] = profile
200                 ct = merged["deployment"].get("compute_target")
201                 canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
202                 if not ct and canonical:
203                     # An active profile with no explicit compute_target (e.g. config.json
blanked it
204                     # to "") gets its canonical target back ? the record never silently lies
about an
205                     # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
206                     merged["deployment"]["compute_target"] = canonical
207                 elif ct and canonical and ct != canonical:
208                     logger.warning(
209                         "[CONFIG] deployment profile %r normally uses compute_target=%r but
"
210                         "config.json overrides it to %r ? honoring the explicit override.",
211                         profile,
212                         canonical,
213                         ct,
214                     )
215                     logger.info(
216                         "[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
217                         profile,
218                         merged["deployment"].get("compute_target"),
219                     )
220             else:
221                 # No profile ? same deep merge so a config.json sub-key overrides its
222                 # default while NEW defaults added in the model since the file was written
223                 # survive ? no silent loss of new knobs on upgrade (D4).
224                 merged = _deep_merge(defaults, file_data)
225                 suggestion = (
226                     suggest_profile()
227                 ) # cheap, env-only (no torch); SUGGESTS only (D15)

```



```

228         if suggestion:
229             logger.debug(
230                 "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
231                 "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
232                 "auto-applies ? D15).",
233                 suggestion,
234             )
235
236     return AppConfig.model_validate(merged)
237
238
239 def save_default_config(path: str | Path | None = None) -> Path:
240     """Write a fresh default config.json.
241
242     Used by scripts/doctor.py diagnostics and the --setup CLI path.
243     Overwrites if the file exists.
244     """
245     cfg_path = Path(path) if path is not None else default_config_path()
246     cfg = AppConfig()
247     cfg_path.parent.mkdir(parents=True, exist_ok=True)
248     cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
249     return cfg_path
250
251
252     # The shipped batteries-included LIGHT-tier default (PACKAGING_PLAN.md P2b / gate 08).
253     # Deliberately MINIMAL: the `truly-local` PROFILE preset (presets.py) supplies
skip_ai_handoff,
254     # local storage and the compute_target, so this stays the handful of knobs a fresh, non-
technical
255     # user gets ? a torch-free, offline, zero-setup experience:
256     # * default_segmenter "motion" ? pure-OpenCV, no torch / GPU / weights / API key
(LIGHT tier);
257     # * profile "truly-local" ? no Gemini handoff, local storage ? never surprise-
bills on
258     # Gemini nor returns 0 rallies on WASB-without-a-key
(08 intent);
259     # * use_gpu false ? honest for a LIGHT (no-GPU) box (motion ignores it
either way).
260     # The first-run wizard (P2c) upgrades this to Gemini once the user supplies a key.
261     LIGHT_DEFAULT_CONFIG: dict[str, Any] = {
262         "sport": "badminton",
263         "deployment": {"profile": "truly-local"},
264         "indexing": {"default_segmenter": "motion", "use_gpu": False},
265     }
266
267
268 def write_light_default_config(
269     path: str | Path | None = None, *, overwrite: bool = False
270 ) -> tuple[Path, bool]:
271     """Seed the batteries-included LIGHT default config (`LIGHT_DEFAULT_CONFIG`).
272
273     Returns `(path, written)`. Unlike :func:`save_default_config` (which dumps a FULL
AppConfig
274     and overwrites), this writes the minimal profile-driven LIGHT bundle and ? by
default ? **never
275     overwrites an existing file**, so a user's edits or the first-run wizard's choices
are preserved
276     (idempotent first-run seeding). Pass ``overwrite=True`` to force. Creates parent
dirs.
277     """
278     cfg_path = Path(path) if path is not None else default_config_path()
279     if cfg_path.exists() and not overwrite:
280         return cfg_path, False

```

```

281     cfg_path.parent.mkdir(parents=True, exist_ok=True)
282     cfg_path.write_text(
283         json.dumps(LIGHT_DEFAULT_CONFIG, indent=2) + "\n", encoding="utf-8"
284     )
285     return cfg_path, True
286
287
288 # Backwards-compatible alias used by some older call sites during transition.
289 def get_default_config() -> dict[str, Any]:
290     """Return defaults as a plain dict (for transitional use only)."""
291     return _get_builtin_defaults()
292

```

6. user (2026-07-08T05:45:09.301Z)

```

1     """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3     A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4     config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5     knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7     Precedence (lowest ? highest), realised by the loader::
8
9         built-in model defaults < profile preset < config.json < env < worker --set
10
11     This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12     module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13     (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14     profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15     today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16     spend is never entered silently).
17     """
18
19     from __future__ import annotations
20
21     import copy
22     import os
23     from typing import Any, Mapping, Optional
24
25     # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26     # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27     PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28     COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30     # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31     # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32     COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33         "truly-local": "local_gpu",
34         "cloud-serving": "cloud_run",
35         "cpu-mock": "cpu_mock",
36     }
37
38     # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39     # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40     # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41     # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,

```

```

42     # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43     PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44         "truly-local": {
45             "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46             # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47             # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48             "indexing": {"skip_ai_handoff": True},
49             "storage": {"backend": "local"},
50         },
51         "cloud-serving": {
52             "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53             # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54             # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid
L4
55             # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
56             # CPU-feed-bound. Serving opts into the measured fast values; local/WSL
unchanged.
57             "indexing": {
58                 "detector_impl": "native",
59                 "skip_ai_handoff": False,
60                 # GPU-resident NVDEC decode (#506/#507): validated on a real L4 (golden gate
PASS,
61                 # F1 0.574 ? baseline 0.582 @ SERVED) AND on the live Cloud Run L4 (the
served
62                 # trajectory reproduced the VM run ? 26479 ? 26481 visible on GX010128),
then rolled
63                 # out + canaried (DEPLOY_REPORT_cloud-serving_gpu-resident-
nvdec_2026-07-07). This makes
64                 # nvdec the code-level serving default so it survives an image rebuild; the
worker's
65                 # WASB_DECODE_BACKEND env was the reversible rollout lever. cuda-only
(validator-enforced).
66                 "wasb": {
67                     "batch_size": 64,
68                     "prefetch_batches": 4,
69                     "decode_backend": "nvdec_resident",
70                 },
71             },
72             "storage": {"backend": "gcs"},
73         },
74         "cpu-mock": {
75             "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
76             # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
77             "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
78             "storage": {"backend": "local"},
79         },
80     }
81
82
83     def is_known_profile(profile: str) -> bool:
84         """True for a non-empty, recognised profile name."""
85         return bool(profile) and profile in PROFILE_PRESETS
86
87
88     def preset_for(profile: str) -> dict[str, Any]:
89         """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
90         module-level table), or ``{}`` for ``""`` / an unknown profile."""
91         return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
92
93
94     def _truthy(val: Optional[str]) -> bool:

```

```

95     """Common env-var truthiness (`1/true/yes/on`, case/space-insensitive)."""
96     if val is None:
97         return False
98     return val.strip().lower() in {"1", "true", "yes", "on"}
99
100
101     def _cuda_available() -> bool:
102         """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
103         and
104         ONLY when a caller opts into `probe_cuda`; any import/runtime failure means 'no
105         GPU'."""
106         try:
107             # Intentionally lazy + local: keeps the config-load path torch-free (torch is
108             heavy
109             # and optional). Only reached when a caller opts into probe_cuda.
110             import torch
111
112             return bool(torch.cuda.is_available())
113         except Exception:
114             return False
115
116     def probe_cuda_available() -> bool:
117         """Public best-effort CUDA probe (lazy torch import; `False` on any failure).
118         Exposed for the per-profile startup checks (`backend/config/startup.py`) so a
119         caller
120         that *wants* the GPU heads-up can pass the result in without the startup module
121         importing
122         torch itself. Best-effort by design: if torch lives only in a detector subprocess
123         env
124         (native WASB), this returns `False` in the main process ? which is why the startup
125         checks
126         treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
127         hard
128         error that could false-fail."""
129         return _cuda_available()
130
131     def suggest_profile(
132         env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
133     ) -> str:
134         """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
135         Signals, in priority order:
136
137         - `K_SERVICE` set (Cloud Run) ? `cloud-serving`
138         - `CI` truthy ? `cpu-mock`
139         - `probe_cuda` and a CUDA GPU present ? `truly-local`
140         - otherwise ? `` (no strong suggestion)
141
142         `probe_cuda` defaults to `False` so the common config-load path stays cheap and
143         never
144         imports torch; a CLI `--detect` or cloud-serving startup that wants the GPU probe
145         passes
146         `probe_cuda=True`. The result is advisory only: the caller decides whether to act
147         on it.
148
149         """
150         e = os.environ if env is None else env
151         if e.get("K_SERVICE"):
152             return "cloud-serving"
153         if _truthy(e.get("CI")):
154             return "cpu-mock"
155         if probe_cuda and _cuda_available():
156             return "truly-local"

```

```
149         return ""
150
```

7. user (2026-07-08T05:45:16.913Z)

```
backend\main.py:268:    # default (deployment.compute_target). Only honoured for these two
values; see _maybe_dispatch_remote.
backend\main.py:401:        "compute_target": getattr(deployment, "compute_target", "") or
"",
backend\orchestration.py:200:    if (getattr(deployment, "compute_target", "") or "") ==
"cloud_run":
backend\orchestration.py:434:    """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path):
when ``deployment.compute_target
backend\orchestration.py:901:        # COMPUTE_DECOUPLED_SERVING M6 ? API: with
deployment.compute_target=cloud_run, run the heavy
backend\compute\__init__.py:4:``ComputeBackend`` ONLY when a non-local
``deployment.compute_target`` is selected, and ``None``
backend\compute\__init__.py:50:    """Return the compute backend for
``config.deployment.compute_target``, or ``None`` to run
backend\compute\__init__.py:53:    Only ``compute_target == "cloud_run"`` returns a non-None
backend (a ``CloudRunCompute``). Its
backend\compute\__init__.py:60:    server's configured default. ``None`` falls back to
``config.deployment.compute_target``.
backend\compute\__init__.py:65:        else getattr(getattr(config, "deployment", None),
"compute_target", "") or ""
backend\compute\cloud_run.py:21:The dispatched container is forced to run **INLINE**
(``compute_target=local_gpu`` + native detector)
backend\compute\cloud_run.py:56:# (compute_target=local_gpu disarms the cmd_infer dispatch
guard; native = the L4 WASB runner.)
backend\compute\cloud_run.py:58:    "deployment.compute_target=local_gpu",
backend\compute\base.py:8:The seam is **default-OFF**: only ``deployment.compute_target ==
"cloud_run"`` selects a
backend\api\job_worker.py:58:    ``deployment.compute_target=cloud_run``, OR the per-request
``compute='cloud'`` override path is
backend\api\job_worker.py:63:    if getattr(dep, "compute_target", "") == "cloud_run":
backend\api\job_worker.py:360:        getattr(getattr(config, "deployment", None),
"compute_target", "") == "cloud_run"
backend\worker\_main__.py:177:    container runs this SAME cmd_infer INLINE (compute_target
forced local_gpu) ? never re-dispatch.
backend\worker\_main__.py:250:    # M6: a cloud compute_target DISPATCHES to a Cloud Run Job
instead of running in-process.
backend\worker\_main__.py:253:    # dispatcher forces compute_target=local_gpu), so it never
recursively re-dispatches.
backend\config\loader.py:200:        ct = merged["deployment"].get("compute_target")
backend\config\loader.py:203:        # An active profile with no explicit compute_target
(e.g. config.json blanked it
backend\config\loader.py:205:        # active profile's compute_target (it stays
meaningful for the M2+ ComputeBackend).
backend\config\loader.py:206:        merged["deployment"]["compute_target"] = canonical
backend\config\loader.py:209:        "[CONFIG] deployment profile %r normally uses
compute_target=%r but "
backend\config\loader.py:216:        "[CONFIG] deployment profile ACTIVE: %s
(compute_target=%s)",
backend\config\loader.py:218:        merged["deployment"].get("compute_target"),
backend\config\loader.py:254:# local storage and the compute_target, so this stays the handful
of knobs a fresh, non-technical
backend\config\presets.py:30:# Each profile maps 1:1 to its canonical compute_target. Used to
apply the preset and to warn
backend\config\presets.py:31:# when config.json explicitly overrides compute_target into a state
inconsistent with the profile.
backend\config\presets.py:39:# that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
backend\config\presets.py:45:    "deployment": {"profile": "truly-local", "compute_target":
"local_gpu"},
backend\config\presets.py:52:    "deployment": {"profile": "cloud-serving",
```

```

"compute_target": "cloud_run"},
backend\config\presets.py:75:         "deployment": {"profile": "cpu-mock", "compute_target":
"cpu_mock"},
backend\config\models.py:471:     compute_target: Literal["", "local_gpu", "cloud_run",
"cpu_mock"] = ""
backend\config\models.py:473:     # bucket-poll CONCURRENTLY when compute_target=="cloud_run".
Each is an I/O-bound Cloud Run Job
backend\ui\assets\index-kJhH4dVE.js:60:[Omitted long matching line]

```

8. user (2026-07-08T05:45:17.900Z)

```

backend\compute\cloud_run.py:268:         for kv in (*spec.config_overrides,
*self._container_overrides):
backend\compute\cloud_run.py:269:         argv += ["--set", kv]
backend\compute\base.py:43:     # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so
the spec stays frozen/hashable.
backend\compute\base.py:44:     config_overrides: tuple[str, ...] = ()

```

9. user (2026-07-08T05:45:22.712Z)

```

1     """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2     for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4     Flow (the cloud-serving compute path):
5     1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6         --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7     2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8         completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9         it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10        cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11        job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12        worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13        the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14        *Fast negative path:* while the marker stays the SOLE success signal, each poll
also probes the
15        Cloud Run Executions API ? if the execution has TERMINATED without ever writing a
marker (a
16        worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
diagnostically
17        (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile
wait.
18    3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
19        ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
20
21    The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
22    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
23
24    The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
25    with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
26    owner-verified on the M7 real run.
27    """

```

```

28
29     from __future__ import annotations
30
31     import json
32     import logging
33     import os
34     import tempfile
35     import uuid
36     from abc import ABC, abstractmethod
37     from typing import Any, Callable, List, Optional
38
39     from backend.compute.base import (
40         ComputeBackend,
41         ComputeJobSpec,
42         ComputeResult,
43         RemoteExecutionFailed,
44     )
45     from backend.infrastructure.execution_policy import (
46         DEFAULT_POLICY,
47         ExecutionPolicy,
48         PolicyTimeout,
49         poll_until,
50     )
51     from backend.storage import fetch, get_storage, parse_uri
52
53     LOG = logging.getLogger("compute.cloud_run")
54
55     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
56     # branch.
57     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
58     # runner.)
59     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
60         "deployment.compute_target=local_gpu",
61         "indexing.detector_impl=native",
62     )
63
64     class CloudRunJobClient(ABC):
65         """Triggers a Cloud Run **Job** execution with per-execution container arg
66         overrides.
67
68         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
69         implementation
70         overrides the job's container ``args`` for this execution and starts it."""
71
72         @abstractmethod
73         def run_job(
74             self,
75             job_name: str,
76             argv: List[str],
77             *,
78             region: str,
79             project: Optional[str] = None,
80         ) -> str:
81             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
82             return an
83             execution id/name (for logs). Does NOT block on completion (the caller bucket-
84             polls)."""
85
86         @abstractmethod
87         def cancel_execution(self, execution: str) -> None:
88             """Cancel a running execution by the name ``run_job`` returned. Called when the
89             bucket-poll
90             times out so the abandoned execution doesn't keep billing the GPU until
91             ``--task-timeout``.

```

```

85         May raise ? the caller treats every failure as best-effort (the original poll
timeout is
86         NEVER masked by a cancel error)."""
87
88     def execution_terminal_state(self, execution: str) -> Optional[str]:
89         """Return the execution's TERMINAL state as a short label (`"Failed"` /
`"Cancelled"` /
90         `"Succeeded"`) if it has finished, else `None` (still pending/running, or
the state
91         can't be determined).
92
93         ADVISORY, not authoritative: it feeds `run_infer`'s fast negative path so a
worker that
94         terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails
fast instead
95         of bucket-polling the full budget. The done-marker stays the ONLY success
signal.
96
97         NON-abstract with a default of `None` (additive / default-OFF): a client that
can't ? or
98         chooses not to ? poll the Executions API degrades transparently to today's
marker-only
99         polling. Implementations MAY raise on a transient API error ? `run_infer`
treats a raise
100        as `None` (unknown ? keep waiting), so a flaky status API never fails a
healthy run."""
101        return None
102
103
104    class GoogleCloudRunJobClient(CloudRunJobClient):
105        """Real client ? lazy-imports `google-cloud-run`. Owner-verified on the M7 real
run; CI uses
106        a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
107
108        def run_job(
109            self,
110            job_name: str,
111            argv: List[str],
112            *,
113            region: str,
114            project: Optional[str] = None,
115        ) -> str:
116            # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
117            # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
118            # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
119            if not project:
120                raise RuntimeError(
121                    "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
122                    "to resolve the job path; set it on the control plane (see
get_compute_backend)."
123                )
124            try:
125                from google.cloud import run_v2 # type: ignore[attr-defined]
126            except Exception as exc: # pragma: no cover - real SDK not present in CI
127                raise RuntimeError(
128                    "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
129                    "control plane (it is intentionally NOT a CI/test dependency)."
130                ) from exc
131            client = (

```



```

132         run_v2.JobsClient()
133     ) # pragma: no cover - exercised only on the real M7 run
134     name = client.job_path(project, region, job_name)
135     overrides = run_v2.RunJobRequest.Overrides(
136         container_overrides=[
137             run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
138         ]
139     )
140     op = client.run_job(
141         request=run_v2.RunJobRequest(name=name, overrides=overrides)
142     )
143     return (
144         getattr(op, "metadata", None)
145         and getattr(op.metadata, "name", "")
146         or "execution"
147     )
148
149     def cancel_execution(self, execution: str) -> None:
150         # run_job falls back to the literal string "execution" when the operation
151         # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
152         # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
153         # (where google-cloud-run is absent).
154         if "/executions/" not in (execution or ""):
155             raise RuntimeError(
156                 f"not a fully-qualified Cloud Run execution name: {execution!r} ?
157                 "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
158                 "one from the operation metadata, so there is nothing to cancel by
159                 name)."
160             )
161         try:
162             from google.cloud import run_v2 # type: ignore[attr-defined]
163         except Exception as exc: # pragma: no cover - real SDK not present in CI
164             raise RuntimeError(
165                 "google-cloud-run is required to cancel a Cloud Run execution; install
166                 "control plane (it is intentionally NOT a CI/test dependency)."
167             ) from exc
168         client = (
169             run_v2.ExecutionsClient()
170         ) # pragma: no cover - exercised only on a real run
171         client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
172
173     def execution_terminal_state(self, execution: str) -> Optional[str]:
174         # Advisory status probe (see the ABC docstring): fetch the execution and
175         # classify it via the
176         # pure, unit-tested ``_classify_execution_state``. A non-resolvable name /
177         # absent SDK returns
178         # None (can't probe ? keep marker-polling) ? unlike run_job/cancel this never
179         # raises on bad
180         # input, because the probe is best-effort and must not fail a healthy run.
181         if "/executions/" not in (execution or ""):
182             return None
183         try:
184             from google.cloud import run_v2 # type: ignore[attr-defined]
185         except Exception: # pragma: no cover - real SDK not present in CI
186             return None
187         client = (
188             run_v2.ExecutionsClient()
189         ) # pragma: no cover - exercised only on a real run
190         exe = client.get_execution( # pragma: no cover - real run only

```

```

187         request=run_v2.GetExecutionRequest(name=execution)
188     )
189     return _classify_execution_state(exe) # pragma: no cover - real run only
190
191
192     def _classify_execution_state(exe: Any) -> Optional[str]:
193         """Map a Cloud Run ``run_v2.Execution`` to a terminal state label (``"Failed"`` /
194         ``"Cancelled"``
195         / ``"Succeeded"``), or ``None`` if it is still pending/running. PURE (no
196         SDK/network) so it is
197         unit-tested against both real proto-plus ``Execution`` objects and lightweight fakes
198         ? the actual
199         ``get_execution`` fetch is the only part that can't run in CI.
200         ``google-cloud-run`` is a **proto-plus** library, so a ``google.protobuf.Timestamp``
201         field is
202         marshalled to a native Python ``datetime`` when set and to ``None`` when unset ? NOT
203         to a proto
204         ``Timestamp`` with a ``.seconds`` field (a ``datetime`` has ``.second``, singular).
205         Terminality is
206         therefore "``completion_time`` is not None"; the per-task counts (plain ints) then
207         classify it,
208         with a cancel taking precedence (a cancelled execution reports ``cancelled_count >
209         0``)."""
210         if getattr(exe, "completion_time", None) is None:
211             return None # no terminal completion timestamp yet ? still pending/running
212         if getattr(exe, "cancelled_count", 0):
213             return "Cancelled"
214         if getattr(exe, "failed_count", 0):
215             return "Failed"
216         if getattr(exe, "succeeded_count", 0):
217             return "Succeeded"
218         return "Failed" # terminal but no positive success ? treat as failed (conservative)
219
220
221     class CloudRunCompute(ComputeBackend):
222         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
223
224         def __init__(
225             self,
226             *,
227             run_client: CloudRunJobClient,
228             job_name: str,
229             region: str,
230             project: Optional[str] = None,
231             storage_config: Optional[dict] = None,
232             policy: ExecutionPolicy = DEFAULT_POLICY,
233             token: Optional[str] = None,
234             container_overrides: Optional[tuple[str, ...]] = None,
235         ) -> None:
236             self._client = run_client
237             self._job_name = job_name
238             self._region = region
239             self._project = project
240             self._storage_config = storage_config or {}
241             self._policy = policy
242             # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
243             collide.
244             # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
245             (tests).
246             self._token = token
247             self._container_overrides = (
248                 _DEFAULT_CONTAINER_OVERRIDES
249                 if container_overrides is None
250                 else tuple(container_overrides)

```

```

242         )
243
244     def run_infer(
245         self,
246         spec: ComputeJobSpec,
247         *,
248         on_wait: "Callable[[float], None] | None" = None,
249     ) -> ComputeResult:
250         out_root = spec.out_uri.rstrip("/")
251         token = self._token or uuid.uuid4().hex
252         done_uri = f"{out_root}/_dispatch/{token}.done"
253         argv: List[str] = [
254             "infer",
255             "--video-uri",
256             spec.video_uri,
257             "--out-uri",
258             spec.out_uri,
259             "--done-uri",
260             done_uri,
261         ]
262         if spec.segmenter:
263             argv += ["--segmenter", spec.segmenter]
264         if spec.skip_validation:
265             # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
266             # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
267             argv.append("--skip-validation")
268         for kv in (*spec.config_overrides, *self._container_overrides):
269             argv += ["--set", kv]
270
271         execution = self._client.run_job(
272             self._job_name, argv, region=self._region, project=self._project
273         )
274         LOG.info(
275             "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
276             self._job_name,
277             execution,
278             done_uri,
279         )
280
281         try:
282             marker = poll_until(
283                 lambda: self._poll_check(done_uri, str(execution or "")),
284                 self._policy,
285                 label="cloud-run-infer",
286                 logger=LOG,
287                 # Live heartbeat (#482): each still-waiting tick reaches the caller so
288                 # job.progress moves during the whole GPU run instead of freezing.
289                 on_tick=on_wait,
290             )
291         except PolicyTimeout as timeout_exc:
292             # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
293             # so it propagates past here uncaught ? the dispatch handler maps it to a
distinct,
294             # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
295             marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
296             video_id = str(marker.get("video_id", ""))
297             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
298             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
299             rc = int(marker.get("returncode", 1))

```

```

300         if not video_id:
301             LOG.warning(
302                 "cloud-run: job=%s completion marker present but empty/unreadable (no "
303                 "video_id) ? treating as failure",
304                 self._job_name,
305             )
306             rc = rc or 1
307             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
308             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
309             LOG.info(
310                 "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
311             )
312             return ComputeResult(
313                 returncode=rc,
314                 outputs_uri=outputs_uri,
315                 video_id=video_id,
316                 report=report,
317                 execution=str(execution or ""),
318             )
319
320     def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
321         """One bucket-poll iteration, hardened with an Executions-API fast negative
322         path.
323
324         The done-marker is the SOLE success signal and is checked FIRST ? a present
325         marker always wins (even if the execution's status later flips), so a worker that wrote a
326         SUCCESS *or a FAILURE* marker resolves through the normal path with its real returncode. Only
327         when the marker is absent do we consult the execution status: a worker that TERMINATED
328         without ever writing a marker (crash / OOM / an old image argparse-aborting on an unknown
329         flag) would otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min)
330         before failing opaquely. Detecting that terminal state collapses the whole class to a prompt,
331         actionable failure.
332
333         Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
334         ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over,
335         no marker will ever appear)."""
336         marker = self._read_marker(done_uri)
337         if marker is not None:
338             return marker # success signal ? authoritative even if status later flips
339             state = self._terminal_execution_state(execution)
340             if state is None:
341                 return None # still pending/running, or status unknown ? keep bucket-
342                 polling
343                 # The execution is terminal but no marker is present. Re-check the marker ONCE
344                 # race where the worker wrote it between our two reads (mirrors the post-timeout
345                 # rescue in _handle_poll_timeout). If it is genuinely absent, the process is gone and no
346                 # marker will EVER appear ? fail fast + diagnostically rather than wait out the whole poll
347                 budget.
348                 marker = self._read_marker(done_uri)
349                 if marker is not None:
350                     return marker
351                 raise RemoteExecutionFailed(
352                     f"cloud-run: job={self._job_name} execution={execution or '<unknown>'}
353                     reached "

```

```

350         f"terminal state {state!r} with NO completion marker ? the worker exited
before "
351         "writing one (crash / OOM / pre-marker abort, or a failed marker upload on
an "
352         "otherwise-Succeeded run). The execution is over, so the marker will never
appear; "
353         "inspect the Cloud Run Job execution log to diagnose: "
354         f"`gcloud run jobs executions describe {execution or '<execution>'}` "
355         f"--region {self._region}`."
356     )
357
358     def _terminal_execution_state(self, execution: str) -> Optional[str]:
359         """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A
raising or
360         flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
361         so any exception ? and an empty/unresolvable execution name ? degrades to
``None`` (unknown
362         ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
363         if not execution:
364             return None
365         try:
366             return self._client.execution_terminal_state(execution)
367         except Exception: # noqa: BLE001 ? advisory probe: never let it kill the
bucket-poll
368             LOG.debug(
369                 "cloud-run: execution status probe failed for %s ? treating as unknown",
370                 execution,
371                 exc_info=True,
372             )
373             return None
374
375     def _handle_poll_timeout(
376         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
377     ) -> dict:
378         """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
379
380         The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed
job's own
381         ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
382         ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
383         is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
384         L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
385         execution name so the operator can verify the state in the GCP console. A cancel
failure
386         NEVER masks the original timeout."""
387         try:
388             late = self._read_marker(done_uri)
389         except Exception: # noqa: BLE001 ? the final check is best-effort too: absence
? failure
390             LOG.warning(
391                 "cloud-run: final post-timeout marker check failed ? treating as
absent",
392                 exc_info=True,
393             )
394             late = None
395         if late is not None:
396             LOG.warning(
397                 "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
398                 "? rescuing the completed result",

```

```

399         self._job_name,
400         execution,
401     )
402     return late
403 try:
404     self._client.cancel_execution(execution)
405 except Exception as cancel_exc: # noqa: BLE001 ? best-effort: never mask the
timeout
406     cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
407     LOG.warning(
408         "cloud-run: best-effort cancel of execution=%s failed: %s",
409         execution,
410         cancel_exc,
411         exc_info=True,
412     )
413 else:
414     cancel_note = "cancel requested"
415     LOG.info("cloud-run: requested cancel of execution=%s", execution)
416     raise PolicyTimeout(
417         f"{timeout_exc} [cloud-run job={self._job_name} "
418         f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
419         "state in the GCP console]"
420     ) from timeout_exc
421
422 # --- storage-backed completion signal (bucket-poll) ---
423 def _read_marker(self, done_uri: str) -> Optional[dict]:
424     """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
425     ref = parse_uri(done_uri)
426     if not get_storage(ref.backend, config=self._storage_config).exists(ref):
427         return None
428     return (
429         self._read_json(done_uri) or {}
430     ) # empty dict still terminates the poll (present == done)
431
432 def _read_json(self, uri: str) -> dict:
433     # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
434     # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
435     # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
436     try:
437         ref = parse_uri(uri)
438         with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
439             dst = os.path.join(td, ref.name or "obj.json")
440             local = fetch(uri, dst, config=self._storage_config)
441             with open(local, "r", encoding="utf-8") as f:
442                 data: Any = json.load(f)
443             return data if isinstance(data, dict) else {}
444     except Exception:
445         LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
446         return {}
447

```

10. user (2026-07-08T05:45:28.314Z)

```

1     """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3     COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4     ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
    ``None``
5     otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte

```

```

6         identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
7     changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
8     """
9
10    from __future__ import annotations
11
12    import dataclasses
13    import os
14    from typing import Any, Optional
15
16    from backend.compute.base import (
17        ComputeBackend,
18        ComputeJobSpec,
19        ComputeResult,
20        RemoteExecutionFailed,
21    )
22    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
23
24    __all__ = [
25        "ComputeBackend",
26        "ComputeJobSpec",
27        "ComputeResult",
28        "RemoteExecutionFailed",
29        "get_compute_backend",
30    ]
31
32
33    def _storage_dict(config: Any) -> dict:
34        storage = getattr(config, "storage", None)
35        if storage is None:
36            return {}
37        if hasattr(storage, "model_dump"):
38            return storage.model_dump() # type: ignore[no-any-return]
39        return dict(storage) if isinstance(storage, dict) else {}
40
41
42    def get_compute_backend(
43        config: Any,
44        *,
45        run_client: Optional[Any] = None,
46        policy: ExecutionPolicy = DEFAULT_POLICY,
47        token: Optional[str] = None,
48        target_override: Optional[str] = None,
49    ) -> Optional[ComputeBackend]:
50        """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
51        in-process (the default-OFF path).
52
53        Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
54        Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
55        / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
56        pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
57
58        ``target_override`` lets a single request force a target (the SPA compute-picker
maps a
59        per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
the
60        server's configured default. ``None`` falls back to
``config.deployment.compute_target``.

```

```

61     """
62     target = (
63         target_override
64         if target_override is not None
65         else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
66     )
67     if target != "cloud_run":
68         return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
69
70     # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
71     selected.
72     from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
73
74     # Derive the poll budget from config UNLESS the caller injected an explicit policy ?
75     # deployment.remote_poll_max_wait_sec must outlast the worker Job's --task-timeout
76     # slow-but-healthy paid GPU run is never abandoned mid-flight (CP-rebuild $4.3; the
77     old
78     # 1800s default undercut real 1080p60 jobs). Identity check on DEFAULT_POLICY keeps
79     an
80     # explicitly-passed policy (tests, callers with bespoke budgets) untouched.
81     if policy is DEFAULT_POLICY:
82         max_wait = getattr(
83             getattr(config, "deployment", None), "remote_poll_max_wait_sec", None
84         )
85         if max_wait:
86             policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=float(max_wait))
87
88     job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
89     region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
90     project = os.environ.get("GOOGLE_CLOUD_PROJECT")
91     client = run_client if run_client is not None else GoogleCloudRunJobClient()
92     return CloudRunCompute(
93         run_client=client,
94         job_name=job_name,
95         region=region,
96         project=project,
97         storage_config=_storage_dict(config),
98         policy=policy,
99         token=token,
100     )

```

11. user (2026-07-08T05:45:28.829Z)

```

1     """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3     COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4     (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
5     DETECT?WINDOW?STITCH
6     core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
7     (cloud-serving) ? without the core ever branching on the profile (D1).
8
9     The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
10    non-local backend. Every other target (``"" / ``local_gpu`` / ``cpu_mock``) keeps the
11    worker's
12    existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
13    inline").
14    """
15
16    from __future__ import annotations
17
18    from abc import ABC, abstractmethod
19    from dataclasses import dataclass, field
20    from typing import Callable, Optional

```



```

18
19
20     class RemoteExecutionFailed(RuntimeError):
21         """A dispatched remote job's execution reached a TERMINAL state without ever writing
its
22         completion marker ? the worker crashed / OOM'd / aborted (e.g. an old image
argparse-rejecting
23         an unknown flag) before the marker push, or the marker upload itself failed on an
otherwise
24         Succeeded run.
25
26         Distinct from ``PolicyTimeout`` (poll budget exhausted while the execution may STILL
be
27         running): this says the execution is DEFINITELY over and no marker will ever
appear, so the
28         dispatcher must stop bucket-polling immediately instead of burning the full
29         ``remote_poll_max_wait_sec`` (~65 min) on a guaranteed-futile wait. The message
carries the
30         job + execution name + terminal state + the ``gcloud`` command to read the execution
log, so
31         the whole "worker crashed before the done-marker" class fails fast AND
diagnostically."""
32
33
34     @dataclass(frozen=True)
35     class ComputeJobSpec:
36         """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
37         from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
38         deterministic regardless of where the job ran."""
39
40         video_uri: str # input (gs://? for cloud; a path/local:// for local)
41         out_uri: str # output ROOT (gs://? bucket for cloud)
42         segmenter: Optional[str] = None
43         # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
44         config_overrides: tuple[str, ...] = ()
45         # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving
control-plane) is
46         # the authoritative validation gate: it runs the SAME validators with ITS resolved
config and only
47         # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video
decode, and
48         # (c) outright WRONG when the worker's config resolves different thresholds ? it can
REJECT a clip
49         # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no
control-plane)
50         # validating exactly as before.
51         skip_validation: bool = False
52
53
54     @dataclass(frozen=True)
55     class ComputeResult:
56         """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
57         (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
58         empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
59         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
60
61         returncode: int
62         outputs_uri: str = ""

```

```

63         video_id: str = ""
64         report: dict = field(default_factory=dict)
65         # The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution
name) ? surfaced
66         # on the job result so "the cloud path actually ran" is independently verifiable
(gcloud run jobs
67         # executions describe <execution>). Empty for the in-process path / backends without
one.
68         execution: str = ""
69
70
71     class ComputeBackend(ABC):
72         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
73         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
74         execution identically."""
75
76         @abstractmethod
77         def run_infer(
78             self,
79             spec: ComputeJobSpec,
80             *,
81             on_wait: "Callable[[float], None] | None" = None,
82         ) -> ComputeResult:
83             """Run ``spec`` to completion and return its ``ComputeResult``.
84
85             ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend is
86             still WAITING on the remote work ? the caller's live-progress heartbeat (#482),
so a
87             long GPU run doesn't look frozen. Implementations without a wait loop may ignore
it."""
88

```

12. user (2026-07-08T05:45:34.997Z)

```

1         """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3         python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4             [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5         python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7         The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8         is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9         are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10        ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11        the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13        This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14        (validator registry ? segmenter ? report), never a forked pipeline.
15        """
16
17        from __future__ import annotations
18
19        import argparse
20        import csv
21        import json
22        import logging
23        import os
24        import re
25        import sys

```

```

26     import tempfile
27     from typing import Any, List
28
29     from backend import storage
30     from backend.compute import ComputeJobSpec, get_compute_backend
31     from backend.config import (
32         StartupCheckError,
33         enforce_startup_checks,
34         load_config,
35         probe_cuda_available,
36     )
37     from backend.infrastructure.database import Database
38     from backend.pipeline.segmenters.base import segmenter_registry
39     from backend.pipeline.validators.base import registry as validator_registry
40     from backend.reporting import emit_indexer_report
41     from backend.results import Failure
42     from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
43     from backend.utils.hashing import compute_video_id
44
45     logger = logging.getLogger("backend.worker")
46
47
48     def _coerce(v: str) -> Any:
49         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
50         str."""
51         low = v.lower()
52         if low in ("true", "false"):
53             return low == "true"
54         for cast in (int, float):
55             try:
56                 return cast(v)
57             except ValueError:
58                 pass
59         return v
60
61     def _apply_overrides(config: Any, sets: List[str]) -> None:
62         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
63         indexing.skip_ai_handoff=true)."""
64         for kv in sets or []:
65             key, sep, val = kv.partition("=")
66             if not sep:
67                 raise ValueError(f"--set expects k=v, got {kv!r}")
68             parts = key.split(".")
69             obj = config
70             for p in parts[:-1]:
71                 obj = getattr(obj, p)
72             setattr(obj, parts[-1], _coerce(val))
73
74     def _write_intervals_csv(segments: List[dict], path: str) -> None:
75         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
76         friendly artifact (same schema as the rally-annotator labels)."""
77         with open(path, "w", newline="") as f:
78             w = csv.writer(f)
79             w.writerow(["start", "end", "shot_count", "ending_reason"])
80             for s in segments:
81                 w.writerow(
82                     [
83                         f"{float(s.get('start_time', 0.0)):.3f}",
84                         f"{float(s.get('end_time', 0.0)):.3f}",
85                         s.get("shot_count", ""),
86                         s.get("ending_reason", s.get("shot_count_confidence", "")),
87                     ]

```

```

88         )
89
90
91     def _write_report_json(
92         video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
93     ) -> None:
94         with open(path, "w") as f:
95             json.dump(
96                 {
97                     "video_id": video_id,
98                     "status": status,
99                     "segment_count": len(segments),
100                     # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
101                     # control plane re-hydrating from outputs/<md5>/ restores the video row
102                     # with its truthful path ? without this, only the segments recover.
103                     "video_uri": video_uri,
104                     "segments": [
105                         {
106                             "start": float(s.get("start_time", 0.0)),
107                             "end": float(s.get("end_time", 0.0)),
108                         }
109                         for s in segments
110                     ],
111                 },
112                 f,
113                 indent=2,
114             )
115
116
117     def _serialize_and_push_outputs(
118         scratch: str,
119         out_uri: str,
120         video_id: str,
121         segments: List[dict],
122         status: str,
123         storage_cfg: dict,
124         video_uri: str = "",
125     ) -> List[str]:
126         """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
127         push both to
128         ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
129
130         Shared by the SUCCESS path and the failure path (``_fail``) so a detector
131         timeout/abort leaves a
132         ``status="failed"`` report.json in the bucket ? the control plane's source of truth
133         (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
134         non-``success``
135         report as "no usable result" ? retry, and the restart-recovery poll waits for
136         report.json to
137         EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``.
138         """
139         out_local = os.path.join(scratch, "outputs", video_id)
140         os.makedirs(out_local, exist_ok=True)
141         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
142         _write_report_json(
143             video_id,
144             segments,
145             status,
146             os.path.join(out_local, "report.json"),
147             video_uri=video_uri,
148         )
149         out_prefix = out_uri.rstrip("/")
150         pushed: List[str] = []
151         for fn in sorted(os.listdir(out_local)):
152             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
153             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)

```

```

149         pushed.append(uri)
150     return pushed
151
152
153     def _run_startup_checks(config: Any) -> bool:
154         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157         (returns True, ZERO work) when no deployment.profile is selected.
158
159         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162         no-op of work, not just of output."""
163         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164         cuda = (
165             probe_cuda_available() if profile else None
166         ) # torch-free on the default-OFF path
167         try:
168             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169             return True
170         except StartupCheckError:
171             return False # already logged at ERROR by enforce_startup_checks
172
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179         A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
180         from the local 2/3), never a raw traceback:
181         * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution
may still be
182         running (the shim best-effort cancels it).
183         * rc 5 ? the execution has TERMINATED without a marker
(``RemoteExecutionFailed``): a worker
184         crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
185         ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
186         from backend.compute import RemoteExecutionFailed
187         from backend.infrastructure.execution_policy import PolicyTimeout
188
189         spec = ComputeJobSpec(
190             video_uri=args.video_uri,
191             out_uri=args.out_uri,
192             segmenter=args.segmenter,
193             config_overrides=tuple(args.set or ()),
194             skip_validation=bool(getattr(args, "skip_validation", False)),
195         )
196         try:
197             result = compute.run_infer(spec)
198         except RemoteExecutionFailed as e:
199             logger.error(
200                 "[worker] remote execution terminated without a completion marker: %s", e

```

```

201         )
202         return 5
203     except PolicyTimeout as e:
204         logger.warning(
205             "[worker] remote compute did not complete (no marker within budget): %s", e
206         )
207         return 4
208     logger.info(
209         "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
210         result.video_id,
211         result.returncode,
212         result.outputs_uri,
213     )
214     return result.returncode
215
216
217     def _write_done_marker(
218         done_uri: str,
219         video_id: str,
220         returncode: int,
221         segment_count: int,
222         status: str,
223         storage_cfg: dict,
224         scratch: str,
225     ) -> None:
226         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
227         for a remote
228         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
229         return."""
230         try:
231             marker = {
232                 "video_id": video_id,
233                 "returncode": returncode,
234                 "segment_count": segment_count,
235                 "status": status,
236             }
237             local = os.path.join(scratch, "_done.json")
238             with open(local, "w", encoding="utf-8") as f:
239                 json.dump(marker, f)
240             storage.put(local, done_uri, config=storage_cfg)
241             logger.info("[worker] wrote completion marker -> %s", done_uri)
242         except Exception as e: # never fail a good run because the marker push hiccuped
243             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
244
245
246     def cmd_infer(args: argparse.Namespace) -> int:
247         config = load_config(args.config) if args.config else load_config()
248         _apply_overrides(config, args.set)
249         if not _run_startup_checks(config):
250             return 2
251
252         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
253         # process.
254         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
255         # through to the
256         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
257         # (the
258         # dispatcher forces compute_target=local_gpu), so it never recursively re-
259         # dispatches.
260         compute = get_compute_backend(config)
261         if compute is not None:
262             return _dispatch_remote(compute, args)
263
264         storage_cfg = config.storage.model_dump()

```

```

260     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
261     os.makedirs(scratch, exist_ok=True)
262     config.output.output_dir = scratch
263
264     # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
265     local_video = storage.fetch(
266         args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
267     )
268     print(f"[worker] pulled {args.video_uri} -> {local_video}")
269
270     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
271     video_id = compute_video_id(local_video)
272
273     # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
274     # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
275     # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
276     # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
277     # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
278     # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
279     # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
280     # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
281     # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
282     if getattr(args, "skip_validation", False):
283         print(
284             "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
285             "gate and already validated this input."
286         )
287         val_results, val_context = [], {}
288     else:
289         val_results, val_context = validator_registry.run_all_with_context(
290             local_video, config
291         )
292     failures = [r for r in val_results if not r.passed]
293     db = Database(os.path.join(scratch, config.output.db_name))
294     db.add_video(
295         video_id,
296         local_video,
297         "failed" if failures else "processed",
298         [r.model_dump() for r in val_results],
299     )
300
301     def _fail(rc: int) -> int:
302         # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
303         # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
304         # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
305         # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
306         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
307         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote

```

```

308         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
309         try:
310             _serialize_and_push_outputs(
311                 scratch,
312                 args.out_uri,
313                 video_id,
314                 [],
315                 "failed",
316                 storage_cfg,
317                 str(getattr(args, "video_uri", "") or ""),
318             )
319         except Exception as e: # never mask the real failure on an artifact-push hiccup
320             logger.warning("[worker] could not push failure artifacts: %s", e)
321         # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
322         if getattr(args, "done_uri", None):
323             _write_done_marker(
324                 args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
325             )
326         return rc
327
328     segments: List[dict] = []
329     segmenter_actual = None
330     segmentation_ran = False
331     status = "failed"
332     try:
333         if failures:
334             print(
335                 "[worker] validation FAILED: "
336                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
337             )
338             return _fail(2)
339         seg_name = args.segmenter or config.indexing.default_segmenter
340         segmenter_cls = segmenter_registry.get(seg_name)
341         if not segmenter_cls:
342             print(
343                 f"[worker] unknown segmenter: {seg_name!r} "
344                 f"(have {list(segmenter_registry.list_available())})"
345             )
346             return _fail(2)
347         segmenter_actual = seg_name
348         result = segmenter_cls(db, config).process_video(
349             video_id, local_video, max_frames=args.max_frames
350         )
351         segmentation_ran = True
352         if isinstance(result, Failure):
353             print(f"[worker] segmenter FAILED: {result.message}")
354             return _fail(3)
355         segments = result.value if hasattr(result, "value") else result
356         status = "success"
357         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
358         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
359         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
360         if not segments:
361             detector_impl = (
362                 os.environ.get("RALLY_DETECTOR_IMPL")
363                 or getattr(config.indexing, "detector_impl", None)
364                 or "auto"
365             )
366             logger.warning(
367                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "

```



```

368         "The detector substrate yielded no usable trajectories/windows ? check
the detector "
369         "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
370         "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
371         "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
372         "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
373         "detections, or the input resolution differs from the tuned proxy
resolution. "
374         "Re-run with -v for the native command + frame counts.",
375         video_id,
376         segmenter_actual,
377         detector_impl,
378     )
379     finally:
380         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
381         emit_indexer_report(
382             db=db,
383             video_id=video_id,
384             video_path=local_video,
385             config=config,
386             val_results=val_results,
387             val_context=val_context,
388             segmenter_requested=args.segmenter,
389             segmenter_actual=segmenter_actual,
390             was_reingest=False,
391             segmentation_ran=segmentation_ran,
392         )
393
394     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
395     pushed = _serialize_and_push_outputs(
396         scratch,
397         args.out_uri,
398         video_id,
399         segments,
400         status,
401         storage_cfg,
402         str(getattr(args, "video_uri", "") or ""),
403     )
404
405     print(
406         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
407     )
408     for u in pushed:
409         print("    ", u)
410
411     # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
412     # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
413     # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
414     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
415     if getattr(args, "done_uri", None):
416         _write_done_marker(
417             args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
418         )
419     return 0
420
421

```

```

422     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
423     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
424     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
425
426     #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-
DD>_<name>.rallies.csv`,
427     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
428     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
429     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
430
431
432     def _label_clip_name(fname: str) -> str:
433         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.
434
435         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
436         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
437
438         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094`` (canonical, date-
stamped)
439         * ``GX020094.rallies.csv`` -> ``GX020094`` (legacy bare ? same
stem)
440         * ``GX020094_proxy.rallies.csv`` -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
441         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
442
443         Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
444         video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
445         """
446         name = fname.replace(".rallies.csv", "").replace(".csv", "")
447         return _LABEL_DATE_PREFIX.sub("", name)
448
449
450     def _probe_video_fps_width(path: str):
451         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
452
453         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
454         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
455         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
456         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
457         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
458         rather than guesses.
459         """
460         import json
461         import subprocess
462
463         try:
464             out = subprocess.check_output(
465                 [
466                     ffprobe_bin(),
467                     "-v",
468                     "error",
469                     "-select_streams",

```

```

470         "v:0",
471         "-show_entries",
472         "stream=r_frame_rate,width",
473         "-of",
474         "json",
475         path,
476     ],
477     stderr=subprocess.DEVNULL,
478 ).decode()
479 st = (json.loads(out).get("streams") or [{}])[0]
480 num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
481 fps = float(num) / float(den or "1")
482 width = float(st.get("width") or 0)
483 if fps > 0 and width > 0:
484     return fps, width
485 except (
486     subprocess.SubprocessError,
487     ValueError,
488     KeyError,
489     OSError,
490     json.JSONDecodeError,
491 ):
492     pass
493 return None
494
495
496 def _resolve_train_detector(args: argparse.Namespace, config: Any):
497     """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
498     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
499     box,
500     stub=CPU mock). Returns the runner, or prints an error + returns None.
501
502     Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
503     the
504     detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
505     has
506     no shuttle detector and is rejected.
507     """
508     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
509
510     seg_cls = segmenter_registry.get(args.segmenter)
511     if not seg_cls:
512         print(
513             f"[worker] unknown segmenter: {args.segmenter!r} "
514             f"(have {list(segmenter_registry.list_available())})"
515         )
516         return None
517     seg = seg_cls(None, config)
518     if not isinstance(seg, TrajectoryHybridSegmenter):
519         print(
520             f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
521             f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
522         )
523         return None
524     try:
525         runner, _ = seg._resolve_runner(config.indexing)
526     except NotImplementedError as e:
527         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
528         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
529         traceback.
530         print(f"[worker] detector not available: {e}")
531         return None
532     ok, msg = runner.healthcheck()
533     if not ok:
534         print(f"[worker] detector environment not ready: {msg}")

```

```

531         return None
532     print(f"[worker] detector ready ({args.segmenter}): {msg}")
533     return runner
534
535
536 def cmd_train(args: argparse.Namespace) -> int:
537     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
538     to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
539
540     Two trajectory sources (the train pipeline + harness are identical either way):
541     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
542         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
543     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
544         DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
545         is the M3.5 "first real training" path; only the trajectory source flips.
546     """
547     import subprocess
548
549     config = load_config(args.config) if args.config else load_config()
550     _apply_overrides(config, args.set)
551     if not _run_startup_checks(config):
552         return 2
553     storage_cfg = config.storage.model_dump()
554
555     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
556     labels_dir = os.path.join(scratch, "labels")
557     traj_dir = os.path.join(scratch, "trajectories")
558     videos_dir = os.path.join(scratch, "videos")
559     os.makedirs(labels_dir, exist_ok=True)
560     os.makedirs(traj_dir, exist_ok=True)
561
562     corpus = args.corpus_uri.rstrip("/")
563     label_uris = sorted(
564         u
565         for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
566         if u.lower().endswith(".csv")
567     )
568     if len(label_uris) < 2:
569         print(
570             f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
571             f"under {corpus}/labels/"
572         )
573         return 2
574
575     # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
576     runner = None
577     video_by_stem: dict = {}
578     if args.trajectory_source == "detector":
579         os.makedirs(videos_dir, exist_ok=True)
580         runner = _resolve_train_detector(args, config)
581         if runner is None:
582             return 2
583         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
584             vname = storage.parse_uri(vuri).name
585             if not vname.lower().endswith(_VIDEO_EXTS):
586                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
587             video_by_stem[os.path.splitext(vname)[0]] = vuri

```

```

588
589 print(f"[worker] trajectory source: {args.trajectory_source}")
590 specs: List[dict] = []
591 for uri in label_uris:
592     fname = storage.parse_uri(uri).name # e.g. 2026-06-22_GX020094.rallies.csv
593     name = _label_clip_name(fname) # -> GX020094 (date prefix stripped)
594     local_label = storage.fetch(
595         uri, os.path.join(labels_dir, fname), config=storage_cfg
596     )
597     if args.trajectory_source == "detector":
598         vuri = video_by_stem.get(name)
599         if not vuri:
600             print(
601                 f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
602             )
603             continue
604         local_video = storage.fetch(
605             vuri,
606             os.path.join(videos_dir, storage.parse_uri(vuri).name),
607             config=storage_cfg,
608         )
609         video_id = compute_video_id(local_video)
610         # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
611         # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
612         meta = _probe_video_fps_width(local_video)
613         if meta is None:
614             print(
615                 f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
616             )
617             continue
618         fps, frame_width = meta
619         traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
620         if not traj:
621             print(
622                 f"[worker] detector produced no trajectory for {name!r} ? skipping."
623             )
624             continue
625         specs.append(
626             {
627                 "name": name,
628                 "trajectory": traj,
629                 "labels": local_label,
630                 "fps": fps,
631                 "frame_width": frame_width,
632             }
633         )
634     else:
635         from backend.pipeline.detectors.stub_runner import (
636             synth_trajectory_from_labels,
637         )
638
639         traj = os.path.join(traj_dir, f"{name}_traj.csv")
640         synth_trajectory_from_labels(
641             local_label, traj, fps=args.fps, frame_width=args.frame_width
642         )
643         specs.append(
644             {
645                 "name": name,
646                 "trajectory": traj,
647                 "labels": local_label,
648                 "fps": args.fps,
649                 "frame_width": args.frame_width,

```

```

650         }
651     )
652
653     if len(specs) < 2:
654         print(
655             f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
656         )
657         return 2
658     manifest_path = os.path.join(scratch, "manifest.json")
659     with open(manifest_path, "w", encoding="utf-8") as f:
660         json.dump(specs, f, indent=2)
661     src_label = (
662         "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
663     )
664     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
665
666     out_dir = os.path.join(scratch, "artifacts")
667     cmd = [
668         sys.executable,
669         "-m",
670         "training.gen0.harness",
671         "--manifest",
672         manifest_path,
673         "--out",
674         out_dir,
675         "--version",
676         args.version,
677     ]
678     if args.floor:
679         floor_local = (
680             storage.fetch(
681                 args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
682             )
683             if "://" in args.floor
684             else args.floor
685         )
686         cmd += ["--floor", floor_local]
687     print("[worker] training:", " ".join(cmd))
688     rc = subprocess.run(cmd).returncode
689     if rc != 0:
690         print(f"[worker] gen0 harness failed (rc={rc})")
691         return rc
692
693     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
694     bundle = os.path.join(out_dir, args.version)
695     out_prefix = args.out_uri.rstrip("/")
696     pushed = []
697     for fn in sorted(os.listdir(bundle)):
698         uri = f"{out_prefix}/weights/{args.version}/{fn}"
699         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
700         pushed.append(uri)
701     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
702     for u in pushed:
703         print("    ", u)
704     return 0
705
706
707 def build_parser() -> argparse.ArgumentParser:
708     p = argparse.ArgumentParser(
709         prog="python -m backend.worker",
710         description="Storage-aware worker: pull ? run pipeline ? push.",
711     )
712     p.add_argument(
713         "-v",

```

```

714         "--verbose",
715         action="store_true",
716         help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
717     )
718     sub = p.add_subparsers(dest="cmd", required=True)
719
720     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
721     pi.add_argument(
722         "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
723     )
724     pi.add_argument(
725         "--out-uri",
726         required=True,
727         help="output prefix (outputs/<video_id>/? is appended)",
728     )
729     pi.add_argument(
730         "--segmenter", default=None, help="default: indexing.default_segmenter"
731     )
732     pi.add_argument(
733         "--config", default=None, help="config.json path (default: ./config.json)"
734     )
735     pi.add_argument(
736         "--scratch", default=None, help="local scratch dir (default: a temp dir)"
737     )
738     pi.add_argument("--max-frames", type=int, default=None)
739     pi.add_argument(
740         "--done-uri",
741         default=None,
742         help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
743         "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
744         "Default-OFF: omit it for the normal local run.",
745     )
746     pi.add_argument(
747         "--set",
748         action="append",
749         default=[],
750         metavar="k=v",
751         help="config override, e.g. --set indexing.skip_ai_handoff=true",
752     )
753     pi.add_argument(
754         "--skip-validation",
755         action="store_true",
756         help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
757         "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
758         "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
759         "`worker infer` CLI still validates.",
760     )
761     pi.set_defaults(func=cmd_infer)
762
763     pt = sub.add_parser(
764         "train",
765         help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
766     )
767     pt.add_argument(
768         "--corpus-uri",
769         required=True,
770         help="prefix containing labels/ (+ videos/ for "
771         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
772     )
773     pt.add_argument(
774         "--out-uri",

```

```

775         required=True,
776         help="output prefix (weights/<version>/ is appended)",
777     )
778     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
779     pt.add_argument(
780         "--trajectory-source",
781         choices=["synth", "detector"],
782         default="synth",
783         help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
784         "detector = REAL shuttle trajectories via the resolved DetectorRunner "
785         "(needs videos/ in the corpus; honours detector_impl).",
786     )
787     pt.add_argument(
788         "--segmenter",
789         default="wasb_hybrid",
790         help="trajectory hybrid whose detector is used for --trajectory-source",
791     )
792     pt.add_argument(
793         "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
794     )
795     pt.add_argument("--config", default=None)
796     pt.add_argument("--scratch", default=None)
797     pt.add_argument(
798         "--fps",
799         type=float,
800         default=30.0,
801         help="synth trajectory fps (detector probes the video)",
802     )
803     pt.add_argument(
804         "--frame-width",
805         type=float,
806         default=1280.0,
807         help="synth trajectory frame width (detector probes the video)",
808     )
809     pt.add_argument("--set", action="append", default=[], metavar="k=v")
810     pt.set_defaults(func=cmd_train)
811     return p
812
813
814     def _setup_logging(verbose: bool) -> None:
815         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
816         diagnostics are
817         VISIBLE ? the worker previously ran with no logging config, so every
818         segmenter/detector log
819         (detector choice, 'Running native WASB inference', availability failures) was
820         silently dropped.
821         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
822         (the CLI
823         entry), not at import, so importing the module never reconfigures a caller's
824         logging."""
825         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
826         library
827         # configured logging at import), which would leave the level at WARNING and re-
828         suppress the
829         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
830         verbosity.
831         logging.basicConfig(
832             level=logging.DEBUG if verbose else logging.INFO,
833             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
834             datefmt="%H:%M:%S",
835             force=True,
836         )

```



```

831     def main(argv=None) -> int:
832         args = build_parser().parse_args(argv)
833         _setup_logging(getattr(args, "verbose", False))
834         return args.func(args)
835
836
837     if __name__ == "__main__":
838         sys.exit(main())
839

```

13. user (2026-07-08T05:45:42.200Z)

```

150         ) # OverflowError on 'inf'/'1e400'
151         except (ValueError, OverflowError):
152             meta["shot_count"] = sc
153             db.add_play_segment(video_id, start, end, metadata=meta)
154             n += 1
155         return n
156
157
158     # Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS box.
A hosted
159     # cloud-serving control-plane bakes NO detector weights into its image (only the worker
Cloud Run Job
160     # GCSFuse-mounts them), so forcing compute="local" for one of these there can never
succeed ? the
161     # guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
deep
162     # "WASB weights not found". (Names are the registered segmenter identities; see
backend/pipeline/segmenters.)
163     _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
"fusion_hybrid"})
164
165
166     def _local_detector_weights_present(cfg, seg_name: str) -> bool:
167         """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
present + readable
168         on THIS box. Mirrors the native/WSL runners' resolution (env override ?
``indexing.<det>.weights_path``)
169         and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
that would
170         otherwise fail deep in the run. Fusion follows its configured substrate (WASB
default / TrackNet).
171         A non-weights segmenter returns True (nothing local to check)."""
172         name = (seg_name or "").strip().lower()
173         if name not in _WEIGHTS_BACKED_SEGMENTERS:
174             return True
175         idx = getattr(cfg, "indexing", None)
176         substrate = (
177             getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
178         ).strip().lower()
179         if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
180             path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
181         else: # wasb_hybrid, or fusion on the (default) WASB substrate
182             path = (
183                 os.environ.get("WASB_WEIGHTS")
184                 or getattr(getattr(idx, "wasb", None), "weights_path", "")
185                 or ""
186             )
187         return bool(path) and os.path.exists(os.path.expanduser(path))
188
189
190     def _cloud_available(cfg) -> bool:
191         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item

```

```

3).
192
193     True when the control plane is wired for Cloud Run: either the configured default
target is
194     already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
195     (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
196     are present so a forced per-request override can actually dispatch + collect a
result.
197
198     Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
199     deployment = getattr(cfg, "deployment", None)
200     if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
201         return True
202     has_job = bool(
203         os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
204     )
205     out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
206     return bool(has_job and out_root)
207
208
209 def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
210     """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
211     minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
212     with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
213     (queue/cold-start), so the very first tick already shows movement."""
214     minutes = int(elapsed_sec) // 60
215     if minutes < 1:
216         return f"segmenting ({seg_name}) on cloud GPU ? starting up"
217     return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
218
219
220 def submit_time_video_id(input_ref, cfg) -> Optional[str]:
221     """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
222     object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
223     deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
224     ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
225     if getattr(input_ref, "backend", "") != "gcs":
226         return None
227     sc = _storage_settings(cfg)
228     if not sc:
229         return None
230     try:
231         st = storage.get_storage("gcs", config=sc).stat(input_ref)
232         return getattr(st, "md5", None) or None
233     except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
234         logger.info(
235             "[API] submit-time md5 unavailable for %s (metadata stat failed)",
236             getattr(input_ref, "uri", input_ref),
237             exc_info=True,
238         )
239     return None
240
241
242 def _cached_process_payload(
243     video_id: str, validation_results, segments, requested_compute
244 ) -> Dict[str, Any]:
245     """The /api/process result for a video whose detection already exists ? ONE shape

```

```

for
246     both producers (the drain's DB-cache short-circuit and the submit-time cache,
#484)."""
247     return {
248         "video_id": video_id,
249         "status": "success",
250         "message": (
251             f"Reused {len(segments)} cached play segments. "
252             "Pass force=true to reprocess this video."
253         ),
254         "results": validation_results or [],
255         "play_segments": segments,
256         "cached": True,
257         "compute": {"path": "cached", "requested": requested_compute},
258     }
259
260
261 def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
262     """Fetch + parse one small JSON object (cloud backends require a real dst ? the
263     ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
264     decide."""
265     ref = storage.parse_uri(uri)
266     with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
267         local = storage.fetch(
268             uri, os.path.join(td, ref.name or "report.json"), config=storage_config
269         )
270         with open(local, encoding="utf-8") as f:

```

14. user (2026-07-08T05:45:42.927Z)

```

420     return cached_process_result(db, cfg, vid, "") is not None
421
422
423 def _maybe_dispatch_remote(
424     cfg,
425     db,
426     req: "ProcessRequest",
427     video_id: str,
428     seg_name: str,
429     val_results,
430     play_wm: int,
431     cand_wm: int,
432     notify,
433 ) -> Optional[Tuple[Dict[str, Any], bool]]:
434     """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
435     ``deployment.compute_target
436     == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
437     intervals into
438     THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
439
440     Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
441     the cloud job
442     actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
443     observability report
444     doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
445     process segmenter
446     (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
447     non-``cloud_run`` target, so
448     the in-process path stays byte-identical)."""
449     from backend.compute import ComputeJobSpec, get_compute_backend
450
451     def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
452         # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
453         (id > play_wm);
454         # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).

```

```

448         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
449         # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
450         # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
451         # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
452         # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
453         return (
454             {
455                 "video_id": video_id,
456                 "status": "failed",
457                 "message": msg,
458                 "results": [r.model_dump() for r in val_results],
459                 "play_segments": [],
460                 "compute": {
461                     "path": "cloud_run" if ran else "not_run",
462                     "requested": req.compute,
463                     "target": "cloud_run",
464                     "dispatched": ran,
465                 },
466             },
467             ran,
468         )
469
470         # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
471         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
472         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
473         choice = (req.compute or "").strip().lower()
474         if choice and choice not in ("local", "cloud"):
475             logger.warning(
476                 "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
477                 req.compute,
478             )
479             choice = ""
480         if choice == "local":
481             # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
482             # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
483             # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
484             # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
485             # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
486             # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
487             # below exactly as before (the weights-present check is the escape hatch).
488             if (
489                 (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
490                 and _cloud_available(cfg)
491                 and not _local_detector_weights_present(cfg, seg_name)
492             ):
493                 return _failed(
494                     f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
495                     "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'.",
496                     False,

```

```

497         )
498         return None # explicit "run on my machine" ? in-process regardless of the
server's target
499     if choice == "cloud":
500         if not _cloud_available(cfg):
501             return _failed(
502                 "cloud-serving was requested but this server isn't configured for it "
503                 "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
504                 False,
505             )
506         compute = get_compute_backend(cfg, target_override="cloud_run")
507     else:
508         compute = get_compute_backend(
509             cfg
510         ) # server default (default-OFF unless cloud_run)
511     if compute is None:
512         return None # default-OFF ? run the in-process segmenter below (unchanged)
513
514     storage_cfg = getattr(cfg, "storage", None)
515
516     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
517     try:
518         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
519     except ValueError as e:
520         return _failed(
521             f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
522         )
523     if input_ref.backend == "local":
524         # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
525         # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
526         # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
527         # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
528         # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
529         # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
530         # default, while a bare/local-looking root can't stage no matter what
input_backend says
531         # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
532         # gives us nothing to build the staged URI under).
533         input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
534         try:
535             root_backend = (
536                 storage.parse_uri(input_root).backend if input_root else "local"
537             )
538         except ValueError: # unsupported scheme in input_root ? nowhere real to stage
to
539             root_backend = "local"
540         if root_backend == "local":
541             return _failed(
542                 "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
543                 "stage local uploads into; a path local to this machine can't be read by
the Cloud "
544                 "Run job.",
545                 False,
546             )
547         local_src = input_ref.key # resolved local path of the upload

```

```

548         staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
549         try:
550             notify("staging upload to bucket")
551             # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
552             # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.
553             storage.put(
554                 local_src,
555                 staged_uri,
556                 config=(
557                     storage_cfg.model_dump() if storage_cfg is not None else None
558                 ),
559             )
560             except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
561                 logger.error(
562                     "[API] cloud-serving: failed to stage %s -> %s: %s",
563                     local_src,
564                     staged_uri,
565                     e,
566                     exc_info=True,
567                 )
568                 return _failed(
569                     f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
570                     False,
571                 )
572             # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
573             # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
574             # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
575             req.video_path = staged_uri
576             input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
577             db.add_video(
578                 video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
579             )

```

15. user (2026-07-08T05:45:54.736Z)

```

580         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
581         if not out_root:
582             return _failed(
583                 "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
584                 "? that's where the Cloud Run job writes the result.",
585                 False,
586             )
587
588         notify("dispatching (cloud_run)")
589         logger.info(
590             "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
591             req.video_path,
592             out_root,
593         )
594         # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
595         # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
596         # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
597         # (player_pool / max_frames are still dropped on the cloud path ? a separate

```

```

documented follow-up.)
598     overrides = []
599     idx_cfg = getattr(cfg, "indexing", None)
600     sk = getattr(idx_cfg, "skip_ai_handoff", None)
601     if sk is not None:
602         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
603     # Forward the resolved WASB serving knobs so the PRESET governs the worker
604     (#506/#507). The
605     # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
606     preset itself ?
607     # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
608     model
609     # defaults (cpu, batch 8) regardless of what the control-plane resolved.
610     decode_backend is
611     # cuda-only (validator-enforced) and requires a worker image that understands
612     --decode-backend
613     # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
614     parity default).
615     wasb_cfg = getattr(idx_cfg, "wasb", None)
616     if wasb_cfg is not None:
617         decode_backend = getattr(wasb_cfg, "decode_backend", None)
618         if decode_backend:
619             overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
620         batch_size = getattr(wasb_cfg, "batch_size", None)
621         if batch_size:
622             overrides.append(f"indexing.wasb.batch_size={batch_size}")
623         prefetch = getattr(wasb_cfg, "prefetch_batches", None)
624         if prefetch:
625             overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
626     # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
627     authoritative
628     # gate ? it already ran the validators with its resolved config just above (only a
629     PASS reaches
630     # here; a failure returned "failed" before dispatch). A worker re-run would decode
631     the video a
632     # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
633     thresholds and
634     # REJECT a clip this gate already passed (observed 2026-07-07: passed at
635     min_blur_threshold=8.0,
636     # then worker-rejected at its default 20.0 after a successful GPU dispatch).
637     spec = ComputeJobSpec(
638         video_uri=req.video_path,
639         out_uri=out_root,
640         segmenter=seg_name,
641         config_overrides=tuple(overrides),
642         skip_validation=True,
643     )
644     try:
645         # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
646         # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
647         # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
648         which
649         # also gives the SPA a stall signal to time out on, instead of a blind wall
650         clock.
651         result = compute.run_infer(
652             spec,
653             on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
654         )
655     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
656         failure, not a 500
657         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
658         return _failed(f"cloud dispatch error: {e}", True)
659     if result.returncode != 0:
660         return _failed(f"cloud job failed (returncode {result.returncode}).", True)
661     if result.video_id and result.video_id != video_id:

```

```

648         # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
649         # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
650         logger.warning(
651             "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
652             "and the worker hash; ingesting under the local id.",
653             result.video_id,
654             video_id,
655         )
656
657     notify("ingesting (cloud_run)")
658     storage_dict = (
659         storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
660         if hasattr(storage_cfg, "model_dump")
661         else {}
662     )
663     try:
664         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
665     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
666         logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
667         return _failed(
668             f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
669             True,
670         )
671
672     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
673     segments = [
674         s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
675     ]
676     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
677     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
678     # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
679     provenance = {
680         "path": "cloud_run",
681         "requested": req.compute,
682         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
683         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
684         "execution": result.execution,
685         "outputs_uri": result.outputs_uri,
686     }
687     logger.info(
688         "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
689         result.execution,
690         provenance["job"],
691         provenance["region"],
692         result.outputs_uri,
693         video_id,
694     )
695     return (
696         {
697             "video_id": video_id,
698             "status": "success",
699             "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
700             "results": [r.model_dump() for r in val_results],
701             "play_segments": segments,
702             "compute": provenance,

```



```

703         },
704         True,
705     )
706
707
708 def _execute_processing(
709     config=None, db=None, req=None, progress=None
710 ) -> Dict[str, Any]:
711     """The full hash ? validate ? segment ? report pipeline for one video.
712
713     Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
714     module-level ``global_config``/``db_instance`` (backward-compatible with existing
715     tests that pass only ``req``). Route handlers always pass explicit params; older
716     test code that calls ``_execute_processing(req)`` still works.
717
718     ``progress`` is an optional callable(stage: str) used to surface coarse job
719     progress.
720
721     Raises on unexpected internal errors ? the caller records those as a job failure
722     """
723     import backend.main as _main # call-time seam: main's globals/model stay patchable
724
725     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
726     # shift positional args ? req arrives in position 0 (config), progress in position 1
727     (db),
728
729     if config is not None and not isinstance(config, AppConfig):
730         # Old signature: _execute_processing(req, progress=None)
731         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
732             req = db
733             db = None
734         elif req is None and isinstance(config, _main.ProcessRequest):
735             req = config
736             config = None
737         # Resolve config/db from module globals when not passed explicitly
738         if config is None:
739             config = _main.global_config
740         if db is None:
741             db = _main.db_instance
742
743     notify = progress or (lambda stage: None)

```

16. user (2026-07-08T05:46:00.637Z)

```

740
741     notify("hashing")
742     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
743     (identity ?
744     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
745     original
746     # req.video_path (the source URI) is kept for the DB record + report; only the file-
747     reading
748     # consumers (hash / validators / segmenter) use the resolved local path.
749     local_video = ""
750     try:
751         local_video = storage.acquire_local_input(
752             req.video_path, getattr(config, "storage", None)
753         )
754         video_id = _main.compute_video_id(local_video)
755     except Exception:
756         storage.cleanup_acquired_local_input(
757             req.video_path, local_video, getattr(config, "storage", None)
758         )
759         raise
760     try:
761         existing_video = db.get_video(video_id)
762     except Exception:

```

```

760         storage.cleanup_acquired_local_input(
761             req.video_path, local_video, getattr(config, "storage", None)
762         )
763         raise
764     was_reingest = existing_video is not None
765
766     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
767     default_seg = getattr(
768         getattr(config, "indexing", None), "default_segmenter", "motion"
769     )
770     seg_name = req.segmenter or default_seg
771     segmenter_actual = None
772     segmentation_ran = False
773     val_results: List[Any] = []
774     val_context: Dict[str, Any] = {}
775     # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
776     # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
777     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
778     compute_actual: Optional[str] = None
779     response: Optional[Dict[str, Any]] = None
780     try:
781         if (
782             existing_video
783             and existing_video.get("status") == "processed"
784             and not req.force
785         ):
786             cached_segments = db.query_play_segments(video_id, {})
787             if cached_segments:
788                 logger.info(
789                     "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
790                     video_id,
791                 )
792             response = _cached_process_payload(
793                 video_id,
794                 existing_video.get("validation_results", []),
795                 cached_segments,
796                 getattr(req, "compute", None),
797             )
798             return response
799
800     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
801     #     UNLESS this exact content already has a cached verdict under the current
validator config.
802     notify("validating")
803     # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
804     # reupload keys the same row. When that content was ALREADY validated under the
SAME validator
805     # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
806     # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
which the
807     # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
808     # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
809     # here). The fingerprint folds in the validation.* config + sport + validator
set + a schema
810     # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).

```

```

811         # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
812         # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
813         # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
814         val_fingerprint = registry.config_fingerprint(config)
815         # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
816         # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
817         # failing the job. The cache is a pure speedup ? never a new failure mode.
818         reused_validation = False
819         if not req.force:
820             try:
821                 cached = db.get_validation_cache(video_id)
822                 if cached is not None and cached.get("fingerprint") == val_fingerprint:
823                     # Replay the stored ValidationResult list; val_context stays {} (no
fresh
824                     # ffmpeg_meta ? the report's media block degrades gracefully to
size-only, the
825                     # deliberate cost of skipping the decode). Segmentation reads
neither of these.
826                     val_results = [
827                         ValidationResult(**r) for r in cached.get("results", [])
828                     ]
829                     reused_validation = True
830                     logger.info(
831                         "[API] Reusing cached validation verdict (passed=%s) for
video_id=%s; "
832                         "validator config unchanged ? skipping the validator suite. "
833                         "force=true re-validates.",
834                         cached.get("passed"),
835                         video_id,
836                     )
837             except Exception: # noqa: BLE001 ? any cache read/replay fault degrades to
a real validation
838                 reused_validation = False
839                 logger.warning(
840                     "[API] validation-cache reuse failed for video_id=%s ? re-
validating.",
841                     video_id,
842                     exc_info=True,
843                 )
844         if not reused_validation:
845             logger.info(f"Running validations for {req.video_path}...")
846             val_results, val_context = registry.run_all_with_context(local_video,
config)
847             failures = [r for r in val_results if not r.passed]
848             if not reused_validation:
849                 # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
850                 # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
851                 db.set_validation_cache(
852                     video_id,
853                     val_fingerprint,
854                     not failures,
855                     [r.model_dump() for r in val_results],
856                 )
857
858             if failures:
859                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
860                 # status; it no longer destroys the video's prior good segments.
861                 db.add_video(

```

```

862         video_id,
863         req.video_path,
864         "failed",
865         [r.model_dump() for r in val_results],
866     )
867     response = {
868         "video_id": video_id,
869         "status": "failed",
870         "message": _validation_failure_message(failures),
871         "results": [r.model_dump() for r in val_results],
872     }
873     return response
874
875     # 2. Run segmenter & indexer
876     logger.info("[API] Video passed checks. Segmenting and indexing...")
877     db.add_video(
878         video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
879     )
880
881     segmenter_cls = segmenter_registry.get(seg_name)
882     if not segmenter_cls:
883         # Submit-time validation already 400s unknown names; this is the defensive
884         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
885         available = list(segmenter_registry.list_available().keys())
886         response = {
887             "video_id": video_id,
888             "status": "failed",
889             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
890             "results": [r.model_dump() for r in val_results],
891             "play_segments": [],
892         }
893         return response
894
895     logger.info(f"[API] Using segmenter: {seg_name}")
896     notify(f"segmenting ({seg_name})")
897     segmenter_actual = seg_name
898     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
899     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
900     play_wm, cand_wm = db.segment_watermarks(video_id)
901     # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
902     # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
903     # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
904     # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
905     # they're dropped on the cloud path in this slice ? a documented follow-up.)
906     remote = _maybe_dispatch_remote(
907         config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
908     )
909     if remote is not None:
910         response, segmentation_ran = (
911             remote # `ran` = did the cloud job actually execute (report)
912         )
913         return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
914     # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
915     compute_actual = "in_process"
916     segmenter = segmenter_cls(db, config)

```

```

917         try:
918             # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
919             # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
920             with _LOCAL_COMPUTE_LANE:
921                 result = segmenter.process_video(
922                     video_id, local_video, req.player_pool, req.max_frames
923                 )
924         except Exception:
925             # A raising segmenter must not leave half-written rows: restore the pre-run
926             # state (same destroy-after-confirm contract as the Failure branch), then
let
927             # the job worker record the failure.
928             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
929             raise
930         segmentation_ran = True
931
932         if isinstance(result, Failure):
933             logger.error(f"[API] Segmenter failed: {result.message}")
934             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
935             response = {
936                 "video_id": video_id,
937                 "status": "failed",
938                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
939                 "results": [r.model_dump() for r in val_results],

```

17. user (2026-07-08T05:46:06.933Z)

```

1      """Startup checks ? per-profile coherence (M5) + exposure-safety posture (M9 hardening).
2
3      A deployment profile (`truly-local` / `cloud-serving` / `cpu-mock`) implies a
coherent
4      runtime. These checks fail/warn fast at process startup ? a loud signal *before* any
5      (expensive) job runs ? when the active profile is inconsistent with the environment:
6
7      * cloud-serving with a non-cloud storage backend ? a hard **error** (the one
config bug we
8      can detect with certainty + zero false-positive risk: a cloud profile that can't
reach a
9      bucket can never serve). This is the README §4.3 "loud error before any job".
10     * a GPU mismatch (e.g. `device='cuda'` on a box where CUDA isn't visible, or
`device='auto'`
11     falling back to CPU) ? a **warning**, never an error: the probe is best-effort
(torch may live
12     only in the detector subprocess env ? see `presets.probe_cuda_available`), and the
detector's
13     own healthcheck is the real authority. A warning surfaces it early without false-
failing.
14     * creds that can't be confirmed (no `GOOGLE_APPLICATION_CREDENTIALS` / not on
Cloud Run) ?
15     a **warning**: Application Default Credentials can come from the metadata server /
gcloud, which
16     we can't probe offline, so the storage backend's own init is the authority; we just
flag it.
17
18     **Exposure-safety posture (server only, `include_exposure=True`)** When the engine is
*exposed*
19     behind an edge gate (the owner flips `security.edge_auth_enabled` on for the
Cloudflare-Access
20     boundary ? the alpha-shareable "tunnel-to-a-box" topology), the gate is only half the
safety story.
21     A half-configured exposure starts *looking* healthy then leaks / 500s per request, so we
verify the
22     rest of the posture at **boot**: the `allowed_video_root` path-jail is set (else an

```

```

authenticated
23     tenant could read an arbitrary local file ? **error**), the CF team-domain + AUD are
present (else
24     no token can verify ? **error**, lifting the current first-request failure to boot), and
the
25     ``auth`` extra is importable (else every request 500s ? **warning**). Only the HTTP
**server** passes
26     ``include_exposure=True``; the worker reads ``--video-uri`` (not untrusted tenant
paths), so it is
27     never "exposed" and stays unaffected.
28
29     **DEFAULT-OFF (both dimensions):** with no profile selected (``profile == ""``) the
profile checks are
30     a strict **no-op** ? exactly the pre-M5 behaviour; with ``edge_auth_enabled=False``
(today's default)
31     the exposure checks are a strict **no-op** too. So nothing changes for today's manual-
knob deployments.
32     ""
33
34     from __future__ import annotations
35
36     import logging
37     import os
38     from dataclasses import dataclass
39     from typing import Any, Mapping, Optional
40
41     LEVEL_ERROR = "error"
42     LEVEL_WARN = "warn"
43
44     # Storage backends that count as "a cloud bucket" for cloud-serving coherence.
45     _CLOUD_STORAGE = ("gcs", "s3")
46     # STRONG env signals that Google ADC *might* resolve. Deliberately NOT
GOOGLE_CLOUD_PROJECT /
47     # CLOUDSDK_CONFIG ? those are often set without usable creds (a plain project id / a
config dir
48     # with no active login), so they would suppress the heads-up falsely. (s3 creds are
intentionally
49     # left to boto3's default chain ? no parallel heads-up, by design.)
50     _GCP_ENV_HINTS = ("GOOGLE_APPLICATION_CREDENTIALS", "K_SERVICE", "GCE_METADATA_HOST")
51
52
53     @dataclass(frozen=True)
54     class StartupCheck:
55         """One per-profile finding. ``level`` is ``error`` (fatal) or ``warn`` (heads-
up)."""
56
57         level: str
58         code: str # stable, greppable code (e.g. "STORAGE_INCOHERENT")
59         message: str
60
61
62     class StartupCheckError(RuntimeError):
63         """Raised by ``enforce_startup_checks`` when an active profile fails a FATAL
check."""
64
65         def __init__(self, failures: list[StartupCheck]):
66             self.failures = failures
67             super().__init__(
68                 "deployment profile startup check failed: "
69                 + "; ".join(f"[{c.code}] {c.message}" for c in failures)
70             )
71
72
73     def _creds_discoverable(env: Mapping[str, str]) -> bool:
74         """True if *some* Google ADC source is hinted by the environment. Best-effort ? a

```

```

False here
75         only downgrades to a WARNING, never a hard failure (the storage init is the
authority)."""
76         return any(env.get(k) for k in _GCP_ENV_HINTS)
77
78
79     def _auth_extra_available() -> bool:
80         """True if PyJWT (the ``auth`` extra) is importable ? probed WITHOUT importing it
(find_spec),
81         so this stays cheap + side-effect-free. Module-level so tests can monkeypatch it
deterministically
82         (whether PyJWT is installed in the test env must not change the WARN's presence)."""
83         try:
84             import importlib.util
85
86             return importlib.util.find_spec("jwt") is not None
87         except (
88             Exception
89         ): # pragma: no cover - find_spec only raises on a broken parent package
90             return False
91
92
93     def _exposure_checks(config: Any, env: Mapping[str, str]) -> list[StartupCheck]:
94         """Exposure-safety posture for an *exposed* HTTP server. Gated on
``security.edge_auth_enabled``
95         (default False ? ``[]`` = strict no-op, today's behaviour). When edge auth is ON ?
the "I am
96         exposing this engine behind the CF-Access gate" signal ? the gate alone isn't
enough: verify the
97         rest of the posture at boot so a half-configured exposure fails fast instead of
leaking / 500-ing
98         per request. Config-only (no IO beyond the ``find_spec`` auth-extra probe); ``env``
is the
99         caller's environment view (so the host-allowlist check is testable)."""
100         sec = getattr(config, "security", None)
101         if not sec or not getattr(sec, "edge_auth_enabled", False):
102             return [] # DEFAULT-OFF: edge auth off ? no posture enforcement (byte-identical
to today).
103
104         out: list[StartupCheck] = []
105
106         # (1) The path-jail is load-bearing once authenticated tenants can hit /api/process:
WITHOUT it,
107         # a tenant can point the engine at an ARBITRARY local file (allowed_video_root unset
= "any local
108         # file", the single-user default). Fatal for an exposed engine.
109         root = getattr(sec, "allowed_video_root", None)
110         if not root or not str(root).strip():
111             out.append(
112                 StartupCheck(
113                     LEVEL_ERROR,
114                     "EXPOSED_NO_VIDEO_ROOT",
115                     "security.edge_auth_enabled is ON (engine exposed) but
security.allowed_video_root is "
116                     "unset ? an authenticated tenant could make /api/process read an
arbitrary local file. "
117                     "Set allowed_video_root to a jail directory before exposing the
engine.",
118                 )
119             )
120
121         # (2) Edge auth cannot verify a token without the team domain + AUD. Today that only
fails on the
122         # FIRST request (auth.resolve_tenant); lift it to BOOT so a misconfigured gate never
starts

```

```

123     # "healthy" then 401s every request.
124     team = (getattr(sec, "cf_team_domain", "") or "").strip()
125     aud = (getattr(sec, "cf_access_aud", "") or "").strip()
126     if not team or not aud:
127         missing = " + ".join(
128             n for n, v in (("cf_team_domain", team), ("cf_access_aud", aud)) if not v
129         )
130         out.append(
131             StartupCheck(
132                 LEVEL_ERROR,
133                 "EXPOSED_EDGE_AUTH_INCOMPLETE",
134                 f"security.edge_auth_enabled is ON but {missing} is not configured ? the
Cf-Access JWT "
135                 "cannot be verified (every request would fail). Set
security.cf_team_domain "
136                 "(e.g. https://<team>.cloudflareaccess.com) and security.cf_access_aud
(the CF Access "
137                 "application AUD tag).",
138             )
139         )
140
141     # (3) The auth extra (PyJWT[crypto]) is lazy-imported by
auth.verify_cf_access_token. If it's
142     # missing, every request 500s at verify time ? a boot WARN beats a per-request 500.
Warning, not
143     # fatal: the auth module's own ImportError is the final authority.
144     if not _auth_extra_available():
145         out.append(
146             StartupCheck(
147                 LEVEL_WARN,
148                 "EXPOSED_AUTH_EXTRA_MISSING",
149                 "security.edge_auth_enabled is ON but PyJWT[crypto] is not importable ?
Cf-Access "
150                 "verification will fail at request time. Install the 'auth' extra (pip
install -e .[auth]).",
151             )
152         )
153
154     # (4) P0c: an exposed engine binds 0.0.0.0, but the Host-header (DNS-rebinding)
guard defaults to
155     # loopback-only when RALLY_ALLOWED_HOSTS is unset (see main._allowed_hosts_from_env)
? it would
156     # then 400 every request the edge forwards with its public Host. Boot WARN beats a
silent 400-all.
157     if not (env.get("RALLY_ALLOWED_HOSTS", "") or "").strip():
158         out.append(
159             StartupCheck(
160                 LEVEL_WARN,
161                 "EXPOSED_HOST_ALLOWLIST_LOOPBACK",
162                 "security.edge_auth_enabled is ON but RALLY_ALLOWED_HOSTS is unset ? the
Host-header guard "
163                 "defaults to loopback-only and will 400 forwarded requests. Set
RALLY_ALLOWED_HOSTS to your "
164                 "public host(s) (or '*' when the edge gate validates the host).",
165             )
166         )
167
168     return out
169
170
171 def run_startup_checks(
172     config: Any,
173     *,
174     cuda_available: Optional[bool] = None,
175     env: Optional[Mapping[str, str]] = None,

```



```

176         include_exposure: bool = False,
177     ) -> list[StartupCheck]:
178         """Return the startup findings for ``config``.
179
180         ``cuda_available`` is the caller's best-effort GPU probe (``None`` = not probed ?
GPU checks
181         are skipped; the API server passes ``None``, the worker passes
``presets.probe_cuda_available()``).
182         ``include_exposure`` adds the exposure-safety posture (``_exposure_checks``) ? the
HTTP **server**
183         passes ``True``; the worker leaves it ``False`` (it is never "exposed"). Both
dimensions are
184         DEFAULT-OFF: no profile ? no profile checks; ``edge_auth_enabled=False`` ? no
exposure checks.
185         Pure: no IO (beyond the ``find_spec`` auth probe), no torch import ? the caller owns
the GPU probe.
186         """
187         e = os.environ if env is None else env
188         checks: list[StartupCheck] = []
189
190         # Exposure-safety posture ? INDEPENDENT of deployment.profile (a tunnel-to-a-box
exposure often
191         # runs with no profile set), gated on edge_auth_enabled (default-OFF). Server-only.
192         if include_exposure:
193             checks.extend(_exposure_checks(config, e))
194
195         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
196         if not profile:
197             return checks # no profile ? only the exposure posture (itself off unless edge
auth is on).
198
199         storage_backend = getattr(getattr(config, "storage", None), "backend", "local")
200         indexing = getattr(config, "indexing", None)
201         detector_impl = getattr(indexing, "detector_impl", "auto")
202         device = getattr(getattr(indexing, "wasb", None), "device", "cuda")
203
204         if profile == "truly-local":
205             # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud
backend here is
206             # an inconsistent (but not fatal) choice worth surfacing.
207             if storage_backend not in ("local", "gdrive"):
208                 checks.append(
209                     StartupCheck(
210                         LEVEL_WARN,
211                         "STORAGE_UNUSUAL",
212                         f"profile 'truly-local' usually uses local/gdrive storage, but
storage_backend="
213                         f"'{storage_backend}'. Reads will go to a remote backend.",
214                     )
215                 )
216             # GPU heads-up (warning only ? the detector healthcheck is the authority; the
probe is
217             # best-effort and may be False even when a subprocess detector env has CUDA).
218             if cuda_available is False:
219                 if device == "cuda":
220                     checks.append(
221                         StartupCheck(
222                             LEVEL_WARN,
223                             "CUDA_NOT_VISIBLE",
224                             "device='cuda' but no CUDA GPU is visible to this process. If
the box truly "
225                             "has no GPU the detector healthcheck will hard-fail ? set
indexing.wasb.device="
226                             "'auto' for the CPU fallback (M4).",
227

```

```

228         )
229         elif device == "auto":
230             checks.append(
231                 StartupCheck(
232                     LEVEL_WARN,
233                     "CPU_FALLBACK",
234                     "device='auto' and no CUDA GPU is visible to THIS process ? auto
may resolve "
235                     "to CPU (much slower). If torch lives only in the detector
subprocess env this "
236                     "can be a false alarm; the runner's DeviceContext/healthcheck is
the authority.",
237                 )
238             )
239
240         elif profile == "cloud-serving":
241             # The ONE fatal coherence check: a cloud profile that can't reach a bucket can't
serve.
242             if storage_backend not in _CLOUD_STORAGE:
243                 checks.append(
244                     StartupCheck(
245                         LEVEL_ERROR,
246                         "STORAGE_INCOHERENT",
247                         f"profile 'cloud-serving' requires a cloud storage backend "
248                         f"('{','.join(_CLOUD_STORAGE)}'), but
storage_backend='{storage_backend}'. "
249                         "A cloud job cannot read/write a local-only backend.",
250                     )
251                 )
252             elif storage_backend == "gcs" and not _creds_discoverable(e):
253                 checks.append(
254                     StartupCheck(
255                         LEVEL_WARN,
256                         "CREDS_UNCONFIRMED",
257                         "no GOOGLE_APPLICATION_CREDENTIALS / Cloud Run env detected; relying
on Application "
258                         "Default Credentials. The GCS backend will fail loudly at init if
ADC can't resolve.",
259                     )
260                 )
261             # The AI-handoff key: with skip_ai_handoff=False the segmenter demands
262             # ${AI_PROVIDER}_API_KEY at rally time and bails to an EMPTY result when it is
263             # missing ? a silent 0-rally "success" AFTER the GPU pass is paid for (observed
on
264             # the first real cloud-serving litmus, 2026-07-03). The env cannot appear later
in
265             # a running container, so this is a zero-false-positive boot-time fatal.
266             skip_ai = bool(getattr(indexing, "skip_ai_handoff", False))
267             provider = str(getattr(config, "ai_provider", "gemini") or "gemini")
268             key_var = f"{provider.upper()}_API_KEY"
269             if not skip_ai and not e.get(key_var):
270                 checks.append(
271                     StartupCheck(
272                         LEVEL_ERROR,
273                         "AI_HANDOFF_KEY_MISSING",
274                         f"profile 'cloud-serving' runs the {provider} handoff "
275                         f"(indexing.skip_ai_handoff=False) but {key_var} is not set ? the
segmenter "
276                         f"would bail to an empty result after the paid GPU pass. Mount the
key "
277                         f"(e.g. a Secret Manager secret) or set
indexing.skip_ai_handoff=True for "
278                         "the fully-local candidates-as-rallies mode.",
279                     )
280                 )

```

```

281         if device == "cpu":
282             checks.append(
283                 StartupCheck(
284                     LEVEL_WARN,
285                     "CLOUD_CPU_DEVICE",
286                     "profile 'cloud-serving' runs on a GPU (L4) but
indexing.wasb.device='cpu' ? "
287                     "detection will not use the GPU.",
288                 )
289             )
290         elif cuda_available is False and device in ("cuda", "auto"):
291             checks.append(
292                 StartupCheck(
293                     LEVEL_WARN,
294                     "CLOUD_CUDA_NOT_VISIBLE",
295                     "profile 'cloud-serving' expects a GPU but no CUDA GPU is visible to
this process.",
296                 )
297             )
298
299     elif profile == "cpu-mock":
300         # The mock profile is for CI/regression ? a real detector here defeats the
purpose.
301         if detector_impl != "stub":
302             checks.append(
303                 StartupCheck(
304                     LEVEL_WARN,
305                     "MOCK_NONSTUB_DETECTOR",
306                     f"profile 'cpu-mock' normally uses detector_impl='stub', but
detector_impl="
307                     f"'{detector_impl}'. A real detector defeats the GPU-free mock.",
308                 )
309             )
310
311     return checks
312
313
314 def enforce_startup_checks(
315     config: Any,
316     *,
317     cuda_available: Optional[bool] = None,
318     env: Optional[Mapping[str, str]] = None,
319     raise_on_error: bool = True,
320     include_exposure: bool = False,
321     logger: Optional[logging.Logger] = None,
322 ) -> list[StartupCheck]:
323     """Run the checks, LOG each (errors at ERROR, warnings at WARNING), and ? unless
324     ``raise_on_error=False`` ? raise ``StartupCheckError`` if any FATAL check tripped.
325
326     ``include_exposure`` (server-only) adds the exposure-safety posture. Returns the
full findings
327     list (so a caller can inspect/surface them). A no-op with no logging and no raise
when both
328     dimensions are off (``run_startup_checks`` returns ``[]``)."""
329     log = logger or logging.getLogger(__name__)
330     checks = run_startup_checks(
331         config,
332         cuda_available=cuda_available,
333         env=env,
334         include_exposure=include_exposure,
335     )
336     for c in checks:
337         if c.level == LEVEL_ERROR:
338             log.error("[STARTUP] %s: %s", c.code, c.message)
339         else:

```

```

340         log.warning("[STARTUP] %s: %s", c.code, c.message)
341     errors = [c for c in checks if c.level == LEVEL_ERROR]
342     if errors and raise_on_error:
343         raise StartupCheckError(errors)
344     return checks
345

```

18. user (2026-07-08T05:46:07.496Z)

```

1     """Configuration package.
2
3     Public API:
4         from backend.config import load_config, AppConfig, Config
5
6     This package is the canonical place for all configuration access.
7     """
8
9     import os
10
11     from .loader import (
12         LIGHT_DEFAULT_CONFIG,
13         load_config,
14         save_default_config,
15         write_light_default_config,
16     )
17     from .models import ( # re-export common ones
18         AppConfig,
19         Config,
20         DeploymentConfig,
21         IndexingConfig,
22         StitchingConfig,
23         StorageConfig,
24         ValidationConfig,
25     )
26     from .presets import PROFILE_PRESETS, probe_cuda_available, suggest_profile
27     from .startup import (
28         StartupCheck,
29         StartupCheckError,
30         enforce_startup_checks,
31         run_startup_checks,
32     )
33
34
35     def default_db_path(config_path: str = "config.json") -> str:
36         """Resolve the DB path from the typed config ? the single shared helper.
37
38         Replaces the copy-pasted ``_default_db_path()`` in ``run_eval.py`` and
39         ``run_regression.py``.
40         """
41         cfg = load_config(config_path)
42         return os.path.join(cfg.output.output_dir, cfg.output.db_name)
43
44
45     __all__ = [
46         "load_config",
47         "save_default_config",
48         "write_light_default_config",
49         "LIGHT_DEFAULT_CONFIG",
50         "default_db_path",
51         "AppConfig",
52         "Config",
53         "DeploymentConfig",
54         "IndexingConfig",
55         "StitchingConfig",
56         "StorageConfig",

```

```

57     "ValidationConfig",
58     "PROFILE_PRESETS",
59     "suggest_profile",
60     "probe_cuda_available",
61     "StartupCheck",
62     "StartupCheckError",
63     "enforce_startup_checks",
64     "run_startup_checks",
65 ]
66

```

19. user (2026-07-08T05:46:13.484Z)

```

deploy\cloudrun\cloudbuild.yaml
deploy\cloudrun\gen_service_yaml.py
deploy\cloudrun\Dockerfile
deploy\cloudrun\post_deploy_smoke.ps1
deploy\cloudrun\README.md

```

20. user (2026-07-08T05:46:15.078Z)

```

deploy\cloudrun\gen_service_yaml.py:28: * `CLOUD_RUN_JOB` is NOT set ? it's a reserved runtime
env name and the manifest is rejected
deploy\cloudrun\gen_service_yaml.py:130:         - name: DEPLOYMENT_PROFILE
deploy\cloudrun\gen_service_yaml.py:134:         - name: GOOGLE_CLOUD_PROJECT
deploy\cloudrun\post_deploy_smoke.ps1:43:# Decode "env" = pass NO decode override, so the Job's
WASB_DECODE_BACKEND env / preset selects it
deploy\cloudrun\README.md:60:**How the knob reaches the worker.** The worker Job runs with NO
`DEPLOYMENT_PROFILE`, so it does
deploy\cloudrun\README.md:67:**Override / rollback lever (no redeploy).** The native runner
reads `WASB_DECODE_BACKEND` FIRST,
deploy\cloudrun\README.md:71:gcloud run jobs update rally-worker --region $REGION --update-env-
vars WASB_DECODE_BACKEND=nvdec_resident
deploy\cloudrun\README.md:72:gcloud run jobs update rally-worker --region $REGION --remove-env-
vars WASB_DECODE_BACKEND # ? forwarded/preset default
deploy\cloudrun\README.md:100:The control plane runs with the **cloud-serving** profile
(`DEPLOYMENT_PROFILE=cloud-serving` ?
deploy\cloudrun\README.md:103:export DEPLOYMENT_PROFILE=cloud-serving
deploy\cloudrun\README.md:104:export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
deploy\cloudrun\README.md:124:(per-instance SQLite state, issue #471), and deliberately does NOT
set `CLOUD_RUN_JOB` (a
deploy\cloudrun\README.md:138: --set-env-vars DEPLOYMENT_PROFILE=cloud-
serving,CLOUD_RUN_REGION=$REGION,GOOGLE_CLOUD_PROJECT=$PROJECT
deploy\cloudrun\README.md:167:- **`GOOGLE_CLOUD_PROJECT` is required** for the real client ?
`GoogleCloudRunJobClient.run_job`

```

21. user (2026-07-08T05:46:21.652Z)

```

1      # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2
3      The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4      dispatches a Cloud Run **Job** execution running this image's worker on an **L4**, then
**bucket-polls**
5      a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
6
7      > **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real
build + push +
8      > run on an L4** is the **M7** owner step, within the **$100/?10k** budget (D17) ?
confirm cmd + hourly
9      > cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does
**not** build this image.
10

```

```

11     ## 0. Prerequisites (owner)
12     - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
    enabled.
13     - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
    Singapore ? see
14         `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15     - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
    `gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17     ## 1. Build + push the image (Artifact Registry)
18     ```bash
19     PROJECT=<gcp-project>; REPO=rally
20     # L4 GPU is region-limited ? use asia-southeast1 (Singapore), NOT asia-south1 (Mumbai).
21     REGION=asia-southeast1
22     gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
23     # `gcloud builds submit` has NO -f flag, so a non-root Dockerfile needs a build config:
24     gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project $PROJECT .
25     ```
26     *(A CUDA + micromamba image is large; the first build is ~8?10 min. Budget-watch per
    D17.)*
27
28     ## 2. Create the Cloud Run **Job** (GPU, scale-to-zero)
29     The proven shape (first real M7 deploy, 2026-07-03): weights staged in a **same-region
    bucket**
30     mounted read-only via GCS FUSE (no weights in the image ? WASB-C3; no init-fetch step
    needed).
31     ```bash
32     gcloud run jobs create rally-worker \
33         --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
34         --region $REGION --gpu 1 --gpu-type nvidia-l4 --no-gpu-zonal-redundancy \
35         --cpu 8 --memory 32Gi \
36         --max-retries 0 --task-timeout 3600 \
37         --set-env-vars WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar \
38         --add-volume name=serving,type=cloud-storage,bucket=$SERVING_BUCKET \
39         --add-volume-mount volume=serving,mount-path=/gcs
40     ```
41     - **`--no-gpu-zonal-redundancy` is REQUIRED on the CLI**, not just a cost choice (it
    also halves
42     the GPU rate ? right for batch jobs): without the flag `gcloud` asks an interactive
    Y/n which,
43     in a non-interactive shell, **hangs forever with zero output and nothing created**. An
    empty
44     `gcloud` that "did nothing" ? suspect a hidden prompt.
45     - 8 vCPU / 32 GiB: x264 encode is CPU-side, so CPU headroom keeps the paid GPU busy.
    `/tmp` is
46     **RAM-backed** ? the pulled input video lives in memory, so size `--memory` for the
    largest
47     expected upload (a 5.8 GB GoPro file ? 6 GiB of tmpfs).
48
49     ## 2a. GPU-resident WASB decode (#506/#507) ? the current serving default
50     The image ships the **py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec==2.1.0** WASB env with
51     `libnvcuvid`/`libnvidia-encode` **baked in** (multi-stage layer from the matching NVIDIA
    driver ?
52     Cloud Run injects the host driver, not these userspace codec libs).
    `indexing.wasb.decode_backend`
53     selects the decode path:
54     - `cpu` (model default) ? the historical cv2 decode; **byte-unchanged** by this work.
55     - `nvdec_resident` ? NVDEC decode + cv2-faithful affine `grid_sample` resize + GPU
    normalize, all
56     on-GPU. **The cloud-serving preset default** (validated: offline golden gate @ SERVED
    F1 0.574 ?
57     baseline 0.582, AND on the live L4 the served GX010128 trajectory reproduced the VM
    run ?
58     26 479 ? 26 481 visible). CUDA-only (config-validator-enforced).

```

```

59
60     **How the knob reaches the worker.** The worker Job runs with NO `DEPLOYMENT_PROFILE`,
so it does
61     **not** apply the cloud-serving preset itself. The control plane (which *does* have the
profile)
62     resolves the config and **forwards**
`indexing.wasb.{decode_backend,batch_size,prefetch_batches}`
63     to the worker as `--set` overrides (`orchestration.py`, alongside `skip_ai_handoff`). So
the
64     **preset is the single source of truth** ? changing `presets.py` + redeploying the
control plane
65     changes serving; no per-Job knob needed.
66
67     **Override / rollback lever (no redeploy).** The native runner reads
`WASB_DECODE_BACKEND` FIRST,
68     so a Job env var wins over the forwarded `--set`:
69     ```bash
70     # force nvdec (or pin cpu) on the Job regardless of the forwarded config; instant
rollback = remove it
71     gcloud run jobs update rally-worker --region $REGION --update-env-vars
WASB_DECODE_BACKEND=nvdec_resident
72     gcloud run jobs update rally-worker --region $REGION --remove-env-vars
WASB_DECODE_BACKEND # ? forwarded/preset default
73     # full revert: pin the prior image by digest
74     gcloud run jobs update rally-worker --region $REGION --image $REGION-
docker.pkg.dev/$PROJECT/$REPO/rally-worker@sha256:<prev>
75     ```
76     ? Forwarding `decode_backend=nvdec_resident` requires a worker image that understands
77     `--decode-backend` (? #507). Roll the **worker image and the control plane together** ?
a new
78     control plane forwarding nvdec to an old worker image argparse-fails (rc 2).
79
80     ? **Same coupling for `--skip-validation` (worker skips redundant re-validation).** The
control
81     plane is the authoritative validation gate ? it runs the validators with its resolved
config and
82     only dispatches on PASS ? so it now passes a **`--skip-validation`** flag on **every**
dispatch,
83     telling the worker not to re-decode + re-validate (which used its own default thresholds
and could
84     reject a clip the gate already passed). This flag is a **new worker CLI arg**: an OLDER
worker image
85     that predates it **argparse-aborts (rc 2) BEFORE writing the done-marker**, so ? unlike
the `--set`
86     knobs, which an old worker at least parses ? the dispatcher's bucket-poll gets no marker
and waits
87     the **full `deployment.remote_poll_max_wait_sec` (~65 min default)** before failing
opaquely as
88     `cloud dispatch error`, on *every* dispatch. So again: **roll the worker image and the
control plane
89     together, worker-first.** Reverse skew (new worker + old control plane) is safe ? the
old control
90     plane simply never sends the flag, and the new worker validates by default.
91
92     **Post-deploy smoke.** After a cutover, confirm the real L4 path with
93     `deploy/cloudrun/post_deploy_smoke.ps1` (run from PowerShell ? see $Gotchas on MSYS arg-
mangling):
94     ```powershell
95     ./deploy/cloudrun/post_deploy_smoke.ps1 -Video gs://khelsutra-rally-
corpus/videos/mahadevpura_smoke_180s.mp4 -Decode cpu # image sanity (fast)
96     ./deploy/cloudrun/post_deploy_smoke.ps1 -Video gs://khelsutra-rally-
corpus/videos/GX010128.MP4 -Decode env # nvdec via preset/env (hard clip)
97     ```
98
99     ## 3. Wire the dispatcher (control plane)

```

```

100     The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
101     `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
102     ```bash
103     export DEPLOYMENT_PROFILE=cloud-serving
104     export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
105     # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
106     # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
107     python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
108     ```
109     **The AI handoff needs its key IN THE JOB env** (`GEMINI_API_KEY` ? mount a Secret
Manager secret
110     on the job), or run fully-local with `--set indexing.skip_ai_handoff=true` (candidate
windows
111     emitted as rallies ? the eval-parity mode). With neither, the segmenter used to bail to
a silent
112     0-rally "success" **after** the paid GPU pass; the cloud-serving startup checks now fail
this at
113     boot (`AI_HANDOFF_KEY_MISSING`, rc 2) before the video is even pulled.
114
115     ### 3a. Control-plane SERVICE deploy ? required flags for reliable async jobs
116
117     The API server (`/api/process`, `/api/compile` ? #329) runs its **job worker IN-
PROCESS**: the
118     request returns `202` and a background thread drains the job (GPU detection dispatches
to the Job;
119     the reel stitch runs on the control plane itself). On Cloud Run that background thread
only makes
120     progress while the instance has CPU, so a scale-to-zero **service** deploy MUST set:
121
122     **The canonical deploy path is the committed manifest generator** (#473) ? it bakes the
123     cloud-serving config.json into a startup wrapper, sets every flag below, pins
`maxScale=1`
124     (per-instance SQLite state, issue #471), and deliberately does NOT set `CLOUD_RUN_JOB`
(a
125     reserved runtime env name that gets a manifest rejected; the code default is already
right):
126     ```bash
127     python deploy/cloudrun/gen_service_yaml.py --edge-auth on -o service.yaml
128     gcloud run services replace service.yaml --region $REGION # from PowerShell on Windows
129     ```
130     `--edge-auth off` is only for the pre-CF identity-token smoke/e2e window; redeploy with
`on`
131     before testers touch the service again. The flag-based equivalent (for reference ? a
manifest
132     round-trips the bash wrapper's metacharacters, flags don't on Windows):
133     ```bash
134     gcloud run deploy rally-control-plane --image <image> --region $REGION \
135     --no-cpu-throttling \ # CPU stays allocated between requests ? the drain thread
finishes
136     --min-instances 1 \ # one always-warm instance drains reliably (cheap CPU;
NOT the D7 GPU pool)
137     --cpu 2 --memory 4Gi --port 8080 --allow-unauthenticated \
138     --set-env-vars DEPLOYMENT_PROFILE=cloud-
serving,CLOUD_RUN_REGION=$REGION,GOOGLE_CLOUD_PROJECT=$PROJECT
139     ```
140     Without `--no-cpu-throttling`, a `/api/compile` returns 202 and then **stalls** ? the
reel never
141     finishes and startup crash-recovery would mark it terminal. `min-instances 1` does NOT
contradict

```



```

142     D7 (that was the ~$500/mo warm **GPU** pool); this is a ~1-vCPU CPU instance. Defense-
in-depth is in
143     the code: `Database.recover_stale_jobs()` **re-queues** a compile interrupted by a
restart (idempotent
144     stitch) rather than failing it, so even an unlucky scale-down mid-stitch self-heals on
the next drain.
145     Reserve `--allow-unauthenticated` for the pre-CF-auth smoke test only; lock ingress to
the CF proxy
146     (M9) before inviting testers. The image ships the `auth` extra (PyJWT[crypto]), so
flipping
147     `security.edge_auth_enabled` on needs no derived image ? the M9 Cf-Access JWT verify
works out of
148     the box.
149
150     ## 4. M7 acceptance (the owner runlog)
151     Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
152     `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
153     rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
154     exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
155
156     ## Gotchas
157     - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
158     + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
159     - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
160     ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
161     - A **failed** Job (validation rc 2 / segmenter rc 3) now writes a **failure marker**
carrying the
162     real non-zero rc, so the dispatcher's poll terminates **FAST + accurately** (no 30-min
hang on a bad
163     video). Only a **hung/crashed** container (no marker at all) makes the bounded poll
time out
164     (`ExecutionPolicy.max_wait_sec`); the worker CLI maps that to a clean **rc 4**, and
the API path
165     surfaces it as a `cloud dispatch error` job failure (not a traceback).
166     - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
167     - **`GOOGLE_CLOUD_PROJECT` is required** for the real client ?
`GoogleCloudRunJobClient.run_job`
168     fails fast with a clear `RuntimeError` if it is unset/empty (it must resolve a real
`job_path`;
169     an unset project would otherwise build a bogus `projects/None/...` name). Export it
(step 3) before
170     dispatching. The guard runs before the `google-cloud-run` import, so it is exercised
in CI even
171     though the SDK isn't a CI/test dependency.
172     - **Hand-rolled `jobs execute --args` with `gs://` URIs:** gcloud's default comma-split
plus its
173     alternate-delimiter syntax both bite ? repeated `--set` tokens trip the comma parser,
and a
174     `^:^\` delimiter splits the `:` inside `gs://`. Use a delimiter that appears nowhere in
the args,
175     e.g. `--args='^@@^-v@@infer@@--video-uri=gs://?'`. (The dispatcher path doesn't have
this
176     problem ? prefer it; this is for one-off diagnostics.) Container-arg overrides must
include the
177     dispatcher's two forced `--set`s: `deployment.compute_target=local_gpu`,
178     `indexing.detector_impl=native`.
179     - **Git Bash (MSYS) on Windows mangles Unix-looking values** before gcloud sees them:
180     `--add-volume-mount ?mount-path=/gcs` became `C:/Program Files/Git/gcs`, and

```

```

181     `--command=/bin/bash` became `C:/Program Files/Git/usr/bin/bash` (the container then
exec-fails).
182     Run gcloud commands with such flags from PowerShell (`MSYS_NO_PATHCONV=1` breaks
gcloud's own
183     wrapper). PowerShell has its own trap: quote any flag value containing commas.
184     - **First execution after an image push** spends minutes in "Container image import in
progress"
185     (visible in `executions describe status.conditions`, NOT in logs) ? that's progress,
not a hang;
186     later executions reuse the imported image.
187

```

22. user (2026-07-08T05:46:27.406Z)

```

1     ""Generate the control-plane Cloud Run *service manifest* (COMPUTE_DECOUPLED_SERVING
M8/M9, #473).
2
3     The A1 alpha deploy drove `rally-control-plane` from a hand-generated YAML that lived in
a
4     session scratchpad ? this commits the generator so the deploy is reproducible from a
clean
5     clone. It emits a Knative service manifest for::
6
7         python deploy/cloudrun/gen_service_yaml.py --edge-auth off -o service.yaml
8         gcloud run services replace service.yaml --region asia-southeast1
9
10    Why a MANIFEST (`services replace`) and not `gcloud run deploy` flags: the container
command is
11    a /bin/bash wrapper carrying shell metacharacters (pipes, redirects, &&) ? on Windows
the gcloud
12    cmd-shim mangles those through `--command/--args`, while a YAML round-trips them
verbatim. Run
13    gcloud from PowerShell, not git-bash (MSYS rewrites `/tmp`-style paths).
14
15    The app is configured through a config.json baked INTO the manifest: the wrapper
base64-decodes
16    it to `/tmp/apphome/config.json` and exports `RALLY_APP_HOME=/tmp/apphome` before
exec'ing the
17    server, so one image serves every config shape (config.json wins over the cloud-serving
preset ?
18    backend/config/loader.py). `--edge-auth` is REQUIRED and explicit: `off` is only for the
19    identity-token smoke/e2e window; redeploy with `on` (the Cf-Access JWT verify, M9)
before the
20    service faces testers again.
21
22    Deliberate manifest choices (learned on the A1 deploy):
23    * `run.googleapis.com/cpu-throttling: "false"` + minScale 1 ? the in-process job
worker drains
24    on a background thread; without always-allocated CPU a 202'd compile stalls (README
$3a).
25    * maxScale DEFAULTS TO 1 ? the jobs/videos DB is per-instance ephemeral SQLite, so a
second
26    instance splits the queue and `/api/jobs/{id}` polls (issue #471, option 1). Raise
it only
27    once state is shared.
28    * `CLOUD_RUN_JOB` is NOT set ? it's a reserved runtime env name and the manifest is
rejected
29    with it; the code default ("rally-worker") is already correct.
30    * No secrets here: bucket/project/aud/team-domain are the same non-secret identifiers
the
31    committed runlogs carry; the Gemini key stays in Secret Manager (the alpha runs
32    `skip_ai_handoff=true` and never needs it).
33
34    Baked config pins (CONTROL_PLANE_DURABLE_REBUILD P1/D5 ? the live rev-00011 posture,
learned on

```

```

35     the 2026-07-05 owner CUJ; before this, regenerating the manifest silently DROPPED both):
36     * `indexing.default_segmenter=wasb_hybrid` ? the engine default is yolo_hybrid, so an
unpinned
37     cloud job ran the wrong segmenter (#479).
38     * `security.max_part_mb=24` ? upload parts must fit under the ~32 MiB HTTP/1 body cap
at the
39     Google Front End (#480); the engine default (95) gets every part 413'd before
FastAPI runs.
40     NOT fixed via `use-http2`: h2c requires the container itself to speak cleartext
HTTP/2,
41     which uvicorn does not (spec D6) ? the annotation would break the service.
42     * `--memory` defaults to 8Gi (spec D2) ? compile still pulls the gs:// source into the
43     RAM-backed /tmp to stitch, so the instance must hold the largest allowed upload with
44     headroom; >8Gi would force 4 vCPU on Cloud Run for no current need.
45     ""
46
47     from __future__ import annotations
48
49     import argparse
50     import base64
51     import json
52     import sys
53
54     DEFAULT_PROJECT = "gen-lang-client-0306026826"
55     DEFAULT_REGION = "asia-southeast1"
56     DEFAULT_SERVICE = "rally-control-plane"
57     DEFAULT_SERVING_BUCKET = "gs://khelsutra-rally-serving"
58     DEFAULT_CF_TEAM_DOMAIN = "https://khelsutra.cloudflareaccess.com"
59     DEFAULT_CF_ACCESS_AUD =
"c980da67ed888fff935fcd913ae1eb4da63f9d65805d6e1c5ca76f92f517082b"
60
61
62     def build_config(args: argparse.Namespace) -> dict:
63         """The app config the wrapper materializes at RALLY_APP_HOME/config.json."""
64         bucket = args.serving_bucket.rstrip("/")
65         return {
66             "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
67             "indexing": {
68                 # WASB-only alpha: candidate windows ARE the rallies; no Gemini key in this
service.
69                 "skip_ai_handoff": True,
70                 # rev-00011 pin (#479): the engine default is yolo_hybrid ? pin the
validated one.
71                 "default_segmenter": args.default_segmenter,
72             },
73             "storage": {
74                 "backend": "gcs",
75                 "output_root": bucket,
76                 "input_backend": "gcs",
77                 "input_root": f"{bucket}/videos",
78             },
79             "security": {
80                 "edge_auth_enabled": args.edge_auth == "on",
81                 "cf_team_domain": args.cf_team_domain,
82                 "cf_access_aud": args.cf_access_aud,
83                 "allowed_video_root": "/tmp/apphome/output",
84                 # rev-00011 pin (#480): parts must clear the ~32 MiB HTTP/1 cap at the GFE.
85                 "max_part_mb": args.max_part_mb,
86             },
87         }
88
89
90     def build_yaml(args: argparse.Namespace) -> str:
91         """Render the Knative service manifest for `gcloud run services replace`."""
92         cfg_b64 = base64.b64encode(

```

```

93         json.dumps(build_config(args), separators=(",", ":")).encode("utf-8")
94     ).decode("ascii")
95     # mkdir BEFORE the redirect (the redirect can't create the directory), then exec so
the
96     # server is PID 1 and Cloud Run's SIGTERM reaches it directly.
97     wrapper = (
98         "mkdir -p /tmp/apphome && "
99         f"printf %s '{cfg_b64}' | base64 -d > /tmp/apphome/config.json && "
100         "export RALLY_APP_HOME=/tmp/apphome && "
101         "exec python3 -m backend.main --server --host 0.0.0.0 --port 8080"
102     )
103     return f"""\
104 apiVersion: serving.knative.dev/v1
105 kind: Service
106 metadata:
107     name: {args.service}
108     labels:
109         cloud.googleapis.com/location: {args.region}
110 spec:
111     template:
112         metadata:
113             annotations:
114                 autoscaling.knative.dev/minScale: "{args.min_instances}"
115                 autoscaling.knative.dev/maxScale: "{args.max_instances}"
116                 run.googleapis.com/cpu-throttling: "false"
117         spec:
118             timeoutSeconds: {args.timeout}
119             containers:
120                 - image: {args.image}
121                   command: ["/bin/bash"]
122                   args: ["-c", "{wrapper}"]
123                   ports:
124                       - containerPort: 8080
125                   resources:
126                       limits:
127                           cpu: "{args.cpu}"
128                           memory: {args.memory}
129                   env:
130                       - name: DEPLOYMENT_PROFILE
131                         value: cloud-serving
132                       - name: CLOUD_RUN_REGION
133                         value: {args.region}
134                       - name: GOOGLE_CLOUD_PROJECT
135                         value: {args.project}
136                       - name: RALLY_ALLOWED_HOSTS
137                         value: "*"
138 """"
139
140
141 def main(argv: "list[str] | None" = None) -> int:
142     p = argparse.ArgumentParser(description=__doc__.splitlines()[0])
143     p.add_argument(
144         "--edge-auth",
145         choices=("on", "off"),
146         required=True,
147         help="REQUIRED and explicit: 'off' only for the identity-token smoke/e2e window;
148 "
149         "redeploy 'on' (M9 Cf-Access verify) before testers touch it again.",
150     )
151     p.add_argument("--project", default=DEFAULT_PROJECT)
152     p.add_argument("--region", default=DEFAULT_REGION)
153     p.add_argument("--service", default=DEFAULT_SERVICE)
154     p.add_argument(
155         "--image",
156         default=None,

```

```

156         help="Container image; default = the project's rally-worker:latest (post-#470 it
157         "carries cloud-run + auth, so the worker image IS the control-plane image).",
158     )
159     p.add_argument("--serving-bucket", default=DEFAULT_SERVING_BUCKET)
160     p.add_argument("--cf-team-domain", default=DEFAULT_CF_TEAM_DOMAIN)
161     p.add_argument("--cf-access-aud", default=DEFAULT_CF_ACCESS_AUD)
162     p.add_argument(
163         "--default-segmenter",
164         default="wasb_hybrid",
165         help="rev-00011 pin (#479): the engine default is yolo_hybrid; the alpha serves
the "
166         "validated wasb_hybrid.",
167     )
168     p.add_argument(
169         "--max-part-mb",
170         type=int,
171         default=24,
172         help="rev-00011 pin (#480): upload parts must fit under Cloud Run's ~32 MiB
HTTP/1 "
173         "request-body cap (the engine default 95 gets 413'd at the edge).",
174     )
175     p.add_argument("--min-instances", type=int, default=1)
176     p.add_argument(
177         "--max-instances",
178         type=int,
179         default=1,
180         help="Default 1: per-instance SQLite state splits across instances (issue
#471).",
181     )
182     p.add_argument("--cpu", type=int, default=2)
183     p.add_argument(
184         "--memory",
185         default="8Gi",
186         help="Default 8Gi (CONTROL_PLANE_DURABLE_REBUILD D2): compile pulls the gs://
source "
187         "into RAM-backed /tmp, so size for the largest allowed upload plus headroom.",
188     )
189     p.add_argument("--timeout", type=int, default=300)
190     p.add_argument("-o", "--output", default="-", help="Output path, or '-' for
stdout.")
191     args = p.parse_args(argv)
192     if args.image is None:
193         args.image = f"{args.region}-docker.pkg.dev/{args.project}/rally/rally-
worker:latest"
194
195     text = build_yaml(args)
196     if args.output == "-":
197         sys.stdout.write(text)
198     else:
199         with open(args.output, "w", encoding="utf-8", newline="\n") as f:
200             f.write(text)
201     return 0
202
203
204 if __name__ == "__main__":
205     raise SystemExit(main())
206

```

23. user (2026-07-08T05:46:35.556Z)

```

30     # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
31     # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
32     # already parallelize their AI handoff internally via their own pools.
33     # EXCEPTION (#466): when the server CAN dispatch to Cloud Run (see

```

```

_cloud_dispatch_possible), a REMOTE
34     # job is only a bucket-poll (I/O-bound; GPU is on ephemeral L4s, WASB-only has no
Gemini), so the drain
35     # scales to deployment.max_concurrent_remote_jobs to overlap those. This does NOT
parallelize local
36     # work: CPU/GPU-heavy in-process detection + the compile stitch serialize via
37     # orchestration._LOCAL_COMPUTE_LANE regardless of the worker count, so the single-box
invariant holds
38     # and compile/local-override jobs stay serial. The count is fixed at first
_get_executor() creation.
39     #
40     # ? O2 RISK (CODE_CLEANUP_AUDIT $3c): a single worker means ONE stuck or hung job
41     # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
42     # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
43     #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
44     #   - detector timeout_sec kills hung WSL/GPU calls
45     #   - Cloud Run poll has a max_wait_sec bound
46     #   - ffmpeg/ffprobe subprocesses use bounded media timeouts
47     # A future slice should add a per-job wall-clock timeout (the executor itself
48     # cannot enforce timeouts on the Thread).
49     _job_executor: Optional[ThreadPoolExecutor] = None
50     # Drain worker count, fixed at first _get_executor() creation. 1 (serial) by default;
configure_drain()
51     # raises it for cloud_run (#466). A module global (not a _get_executor arg) so the many
call sites +
52     # the tests that monkeypatch `_get_executor` as a no-arg lambda are unchanged.
53     _drain_max_workers: int = 1
54
55
56     def _cloud_dispatch_possible(config: Any = None) -> bool:
57         """True if this server CAN dispatch a job to Cloud Run ? either the profile sets
58         ``deployment.compute_target=cloud_run``, OR the per-request ``compute='cloud'``
override path is
59         wired (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT`` env + an output bucket). Mirrors
60         orchestration._cloud_available so drain concurrency tracks the ACTUAL remote-
dispatch capability,
61         not just the server default (so compute='cloud' overrides on a local-default server
parallelize too)."""
62         dep = getattr(config, "deployment", None)
63         if getattr(dep, "compute_target", "") == "cloud_run":
64             return True
65         if os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT"):
66             stor = getattr(config, "storage", None)
67             if (getattr(stor, "output_root", "") or "").strip():
68                 return True
69         return False
70
71
72     def _resolve_drain_workers(config: Any = None) -> int:
73         """Drain concurrency (#466): a server that CAN dispatch to Cloud Run (see
_cloud_dispatch_possible)
74         may run ``deployment.max_concurrent_remote_jobs`` drain workers so I/O-bound remote
dispatch/poll
75         OVERLAPS. This does NOT parallelize CPU/GPU-heavy LOCAL work ? in-process detection
and the compile
76         stitch serialize via orchestration._LOCAL_COMPUTE_LANE regardless of the worker
count ? so the
77         single-box GPU + rate-fragile Gemini invariant holds, and compile/local-override
jobs stay serial.
78         A server with no remote capability stays serial (1). Defaults to 1 when config is
absent
79         (tests / non-server) ? behaviour unchanged."""
80         if _cloud_dispatch_possible(config):
81             dep = getattr(config, "deployment", None)
82             return max(1, int(getattr(dep, "max_concurrent_remote_jobs", 1) or 1))

```

```

83         return 1
84
85
86     def configure_drain(config: Any = None) -> None:
87         """Set the drain worker count from ``config`` BEFORE the executor is created (#466).
Call once at
88         server startup. No-op effect once the executor already exists (the count is fixed at
creation)."""
89         global _drain_max_workers
90         _drain_max_workers = _resolve_drain_workers(config)
91
92
93     def _get_executor() -> ThreadPoolExecutor:
94         """Lazy module-global executor; tests monkeypatch this for inline execution. Worker
count is
95         ``_drain_max_workers`` (1 by default; raised for cloud_run via ``configure_drain``,
#466)."""
96         global _job_executor
97         if _job_executor is None:
98             _job_executor = ThreadPoolExecutor(
99                 max_workers=_drain_max_workers, thread_name_prefix="jobworker"
100             )
101         return _job_executor
102
103
104     class JobWaiting(Exception):
105         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
106         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`,
107         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
108         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
109         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
110
111         The wiring is additive: the production segmenter path never raises this today (zero
112         behaviour change), so a job runs exactly as before. The hook is what a later slice
113         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
114         """
115
116         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
117             super().__init__(f"job parked for {delay_sec:.0f}s")
118             self.delay_sec = max(0.0, float(delay_sec))
119             self.progress = progress
120
121
122     def _job_to_request(job: Dict[str, Any]):
123         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
124         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
125         the original submit. The row stores exactly the ProcessRequest fields."""
126         import backend.main as _main
127
128         raw_params = job.get("params")
129         params = (

```

24. user (2026-07-08T05:46:36.105Z)

```

330     #: Resume-poll cadence. Module-level so tests drive the loop with a zero-wait cadence.
331     _RESUME_POLL_EVERY_SEC = 15.0
332
333
334     def resume_interrupted_remote_jobs(config: Any, db: Any) -> int:
335         """Re-arm the durable done-signal poll for md5-keyed RUNNING process jobs a restart
336         orphaned (the rows ``recover_stale_jobs`` deliberately left alone). Call once at
startup,

```

```

337     after ``recover_stale_jobs`` and before any drain. Each job resumes on its own
daemon
338     thread (bounded by the handful of rows a single-instance CP can have in flight);
rows are
339     NEVER re-dispatched from here. Returns the number of resumes started.
340
341     On a box that cannot dispatch to Cloud Run at all (the appliance), no remote worker
can
342     exist ? the rows get today's honest terminal failure instead.""
343     rows = db.interrupted_remote_process_jobs()
344     if not rows:
345         return 0
346     if not _cloud_dispatch_possible(config):
347         for job in rows:
348             db.mark_job_failed(str(job["id"]), "interrupted by server restart")
349             logger.warning(
350                 "[JOBS] %d interrupted md5-keyed process job(s) failed: this server cannot "
351                 "dispatch to Cloud Run, so no remote worker can be in flight.",
352                 len(rows),
353             )
354         return 0
355     # Prove each row was actually CLOUD-dispatched (#494 review round 2): an explicit
356     # compute='cloud' always was; NULL means "server default", which is remote ONLY when
the
357     # default target itself is cloud_run (on a default-local server with cloud-override
358     # capability, a NULL row ran in-process ? no remote worker exists to resume).
359     default_is_cloud = (
360         getattr(getattr(config, "deployment", None), "compute_target", "") ==
"cloud_run"
361     )
362     started = 0
363     for job in rows:
364         # Same normalization the execution path applies (_maybe_dispatch_remote
lowercases
365         # req.compute) ? "LOCAL"/" cloud " must mean at recovery exactly what they meant
at run.
366         requested = (str(job.get("compute") or "")).strip().lower()
367         if requested != "cloud" and not default_is_cloud:
368             db.mark_job_failed(str(job["id"]), "interrupted by server restart")
369             continue
370         t = threading.Thread(
371             target=_resume_one_remote_job,
372             args=(config, db, dict(job)),
373             daemon=True,
374             name=f"resume-{str(job['id'])[:8]}",
375         )
376         t.start()
377         started += 1
378     if started:
379         logger.warning(
380             "[JOBS] re-armed the durable done-signal poll for %d interrupted remote
job(s).",
381             started,
382         )
383     return started
384
385
386     def _resume_one_remote_job(config: Any, db: Any, job: Dict[str, Any]) -> None:
387         """One orphaned job's resume: poll ``outputs/<md5>/report.json`` (bounded by the
same
388         ``deployment.remote_poll_max_wait_sec`` budget a live dispatch gets), then complete
or
389         fail the job HONESTLY from what the bucket says. Never dispatches."""

```



```

240     _ingest_remote_segments,
241     _maybe_dispatch_remote,
242     _remote_input_size_bytes,
243     _report_config_for_input,
244     _resolve_compile,
245     read_path_rehydrate,
246     signed_reel_url,
247 )
248
249
250 # Legacy thin wrapper for any remaining direct calls during transition.
251 # New code should import load_app_config / AppConfig from backend.config directly.
252 def load_config(config_path: str = "config.json") -> Dict[str, Any]:
253     cfg = load_app_config(config_path)
254     return cfg.model_dump(mode="json")
255
256
257 # --- API Models ---
258 class ProcessRequest(BaseModel):
259     video_path: str
260     player_pool: Optional[List[str]] = None
261     max_frames: Optional[int] = None
262     segmenter: Optional[str] = None
263     # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
264     # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
265     locale: Optional[str] = None
266     # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
267     # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
268     # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
269     compute: Optional[str] = None
270     # Explicit opt-in reprocess path. By default, a video whose content hash already has
stored
271     # play_segments is returned from cache so refresh/Try again cannot re-bill paid
segmenters.
272     force: bool = False
273
274
275 class AnnotationsRequest(BaseModel):
276     # Finished rally labels for a video, as the annotator's byte-compatible CSV (P8,
from the Studio
277     # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is
written verbatim.
278     video_id: str
279     csv: str
280
281
282 class AssetRequest(BaseModel):
283     # "Store this video in my medium" (STUDIO_MEDIA_LIFECYCLE.md P4, D14). Exactly ONE
source:
284     #   uploaded_path ? a local file from the chunked upload (the durable copy's source;
kept as the
285     #   local working copy for the run), OR
286     #   source_uri ? a video ALREADY at rest in a medium (a pasted
s3:///gcs:///gdrive:/// URI) ?
287     #   registered as-is, no copy (D14b).
288     # medium_uri is the destination store (a bucket/Drive/local folder URI). The asset
is keyed by
289     # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content
hash.
290     uploaded_path: Optional[str] = None

```

```

291     source_uri: Optional[str] = None
292     medium_uri: str
293     sport: Optional[str] = None
294
295
296     class QueryRequest(BaseModel):
297         video_id: str
298         server: Optional[str] = None
299         receiver: Optional[str] = None

```

26. user (2026-07-08T05:46:45.198Z)

```

380     # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
381     # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the
382     # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
383     nvidia_smi = shutil.which("nvidia-smi")
384     return {
385         "api_version": API_VERSION,
386         "engine_version": _engine_version(),
387         "segmenters": sorted(segmenter_registry.list_available().keys()),
388         "default_segmenter": getattr(indexing, "default_segmenter", None),
389         "ffmpeg": {
390             "ffmpeg": bool(ffmpeg),
391             "ffprobe": bool(ffprobe),
392             "ffmpeg_path": ffmpeg,
393             "ffprobe_path": ffprobe,
394         },
395         "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
396         "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
397         "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
398         **uploads_api.upload_caps(cfg), # #480 item 5: the SPA pre-flight reads
max_upload_mb
399         "deployment": {
400             "profile": getattr(deployment, "profile", "") or "",
401             "compute_target": getattr(deployment, "compute_target", "") or "",
402             # item 3: can this server satisfy a per-request compute="cloud" dispatch?
The SPA shows
403             # the "run in the cloud" toggle only when true (else it's local-only).
404             "cloud_available": _cloud_available(cfg),
405         },
406         # P3: which storage media THIS box can store INTO (local always; s3/gcs if
SDK+config;
407         # gdrive if a mount resolves). The P5 medium picker renders only usable
backends. No secrets.
408         "storage": storage_capabilities(
409             getattr(cfg, "storage", None), local_free_path=output_base(cfg)
410         ),
411     }
412
413
414     def _friendly_video_name(filepath: str) -> str:
415         """A human name for a video from its stored path/URI ? the filename stem
(``~/match_2026.mp4`` ?
416         ``match_2026``). Works for a local path, a bucket key, or a ``gdrive://`` URI
(rsplit on ``/``).
417         Falls back to the whole ``filepath`` if there's no basename."""
418         base = str(filepath or "").rstrip("/").rsplit("/", 1)[-1].rsplit("\\", 1)[-1]
419         stem = os.path.splitext(base)[0]
420         return stem or base or str(filepath or "")
421
422
423     def _video_dims(validation_results: Any) -> "tuple[Optional[float], Optional[str]]":

```

```

424         """Pull ``(fps, resolution)`` from the persisted validation results, or ``(None,
None)``. The
425         ``resolution_fps_check`` validator already probed fprobe and stored
``details={width,height,fps}``
426         on its result (every branch) ? so the videos row carries it with no schema change /
re-probe."""
427         for r in validation_results or []:
428             if isinstance(r, dict) and r.get("validator_name") == "resolution_fps_check":
429                 d = r.get("details") or {}

```

27. user (2026-07-08T05:46:54.516Z)

```

backend\main.py:17:# PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is
set (the packaged
backend\main.py:377:         os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "")
or ""
backend\main.py:1406:     # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE
load_app_config reads
backend\main.py:1430:     # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME);
with the env
backend\main.py:1467:         f"use a different --output-dir (RALLY_APP_HOME) to run an
independent instance."
backend\orchestration.py:183:         os.environ.get("WASB_WEIGHTS")
backend\api\uploads.py:70:     ) # P0a: app-home-rooted (CWD-identical when RALLY_APP_HOME
unset)
backend\config\models.py:88:     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a
config-free GPU box.
backend\config\models.py:209:     # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
backend\config\presets.py:65:         # WASB_DECODE_BACKEND env was the reversible rollout
lever. cuda-only (validator-enforced).
backend\config\loader.py:33:# ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a) and equals
``Path("config.json")`` (CWD-relative) when
backend\config\loader.py:36:# from a later RALLY_APP_HOME.)
backend\worker\__main__.py:9:are byte-for-byte unchanged. Combine with
``RALLY_DETECTOR_IMPL=stub`` and
backend\worker\__main__.py:362:         os.environ.get("RALLY_DETECTOR_IMPL")
backend\worker\__main__.py:372:         "runner UNAVAILABLE (weights/repo/WASB_PYTHON
env), the WASB env produced no "
backend\worker\__main__.py:498:     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl:
auto=WSL, native=GPU box,
backend\infrastructure\database.py:115:         f"a different data directory
(RALLY_APP_HOME). DB: {self.db_path}"
backend\pipeline\segmenters\trajectory_hybrid.py:81:         ``RALLY_DETECTOR_IMPL`` env var
(takes precedence) or ``indexing.detector_impl``:
backend\pipeline\segmenters\trajectory_hybrid.py:93:
os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.detector_impl or "auto"
backend\pipeline\segmenters\trajectory_hybrid.py:244:         # _resolve_runner honours the CPU-
stub override (RALLY_DETECTOR_IMPL /
backend\utils\app_home.py:10:     * ``RALLY_APP_HOME`` env set ? that directory is the app home
(the packaged launcher sets it).
backend\utils\app_home.py:11:     * ``RALLY_APP_HOME`` unset ? ``Path(".")`` i.e. the CWD,
exactly the pre-P0a behaviour.
backend\utils\app_home.py:15:calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the
resolver opt-in means importing
backend\utils\app_home.py:30:APP_HOME_ENV = "RALLY_APP_HOME"
backend\utils\app_home.py:39:     ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise
``Path(".")`` ? the CWD, byte-identical
backend\utils\app_home.py:56:     With ``RALLY_APP_HOME`` unset this is ``Path("config.json")``
(CWD-relative), unchanged.
backend\utils\app_home.py:70:     Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is
set AND that file exists. With the
backend\utils\app_home.py:86:     With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM
(the default ``"output"`` when
backend\utils\app_home.py:105:     ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).

```

```

backend\pipeline\detectors\native_wasb_runner.py:17:    (`WASB_WEIGHTS` / `WASB_REPO_DIR` /
`WASB_PYTHON` / `WASB_SCRATCH`), then
backend\pipeline\detectors\native_wasb_runner.py:87:    weights_path: str = "" # REQUIRED (env
WASB_WEIGHTS or indexing.wasb.weights_path)
backend\pipeline\detectors\native_wasb_runner.py:103:    # GPU-resident NVDEC decode (#506,
indexing.wasb.decode_backend / WASB_DECODE_BACKEND).
backend\pipeline\detectors\native_wasb_runner.py:119:    python_bin=env("WASB_PYTHON")
or w.python_bin,
backend\pipeline\detectors\native_wasb_runner.py:120:
weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
backend\pipeline\detectors\native_wasb_runner.py:122:    device=env("WASB_DEVICE") or
w.device,
backend\pipeline\detectors\native_wasb_runner.py:132:
decode_backend=env("WASB_DECODE_BACKEND") or w.decode_backend,
backend\pipeline\detectors\native_wasb_runner.py:204:    "WASB weights path is empty
? set WASB_WEIGHTS (or indexing.wasb.weights_path)."
backend\pipeline\detectors\native_wasb_runner.py:231:    f"python binary not found:
{self.cfg.python_bin!r} (set WASB_PYTHON).",
backend\pipeline\detectors\native_wasb_runner.py:360:    "python binary not found:
%r (set WASB_PYTHON).", self.cfg.python_bin
backend\pipeline\detectors\stub_runner.py:20: ``indexing.detector_impl="stub`` or the
``RALLY_DETECTOR_IMPL=stub`` env var; the real

```

28. user (2026-07-08T05:47:01.592Z)

```

80         defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
81         """
82
83         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
84         python_bin: str = (
85             "python" # the `wasb` conda env python (invoked by path, no activate)
86         )
87         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
88         sport: str = "badminton"
89         device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
90         scratch_dir: str = "" # frame-extraction scratch (env); "" = next to the output dir
91         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no timeout)
92         keep_frames: bool = (
93             False # retain the decoded frame cache after success (debug only)
94         )
95         # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
96         # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
97         # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
98         stream_video: bool = True
99         # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
100         # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
101         batch_size: int = 0
102         prefetch_batches: int = 0
103         # GPU-resident NVDEC decode (#506, indexing.wasb.decode_backend /
WASB_DECODE_BACKEND).
104         # "cpu" (default) = omit the flag ? wasb_infer.py's historical cv2 decode paths,
argv
105         # byte-identical. "nvdec_resident" = decode+resize+normalize on-GPU (torch-2.x env
only).
106         decode_backend: str = "cpu"
107
108         @classmethod
109         def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
110             if isinstance(idx_cfg, dict):
111                 idx_cfg = IndexingConfig(**idx_cfg)

```

```

112         w = idx_cfg.wasb
113         env = os.environ.get
114         # stream_video is tri-state in config: None/absent => native default (stream);
an
115         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
116         sv = w.stream_video
117         return cls(
118             repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
119             python_bin=env("WASB_PYTHON") or w.python_bin,
120             weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
121             sport=w.sport,
122             device=env("WASB_DEVICE") or w.device,
123             scratch_dir=env("WASB_SCRATCH")
124             or env("RALLY_SCRATCH")
125             or env("TMPDIR")
126             or "",
127             timeout_sec=w.timeout_sec,
128             keep_frames=w.keep_frames,
129             stream_video=(True if sv is None else bool(sv)),
130             batch_size=int(w.batch_size or 0),
131             prefetch_batches=int(w.prefetch_batches or 0),
132             decode_backend=env("WASB_DECODE_BACKEND") or w.decode_backend,
133         )
134
135
136     class NativeWasbRunner(DetectorRunner):
137         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
138
139         def __init__(self, cfg: NativeWasbConfig):

```

29. user (2026-07-08T05:47:02.897Z)

SESSION_HANDOFF.md:40:- SD is a replacement compile mode, not an additional output: if selected, build

tests\test_async_jobs.py:362: destroy-after-confirm replacement safety."""

docs\archives\roora-vendor\R0ORA_LABELING_GUIDELINES.md:192:**Boundary placement is the non-negotiable part:**

docs\archives\roora-vendor\R0ORA_LABELING_GUIDELINES.md:342:- `forced\_error` = the opponent's shot CAUSED the miss (pace, placement, deception; the player barely

backend\features\rally_seq.py:6:59.94fps than 30fps) is exactly the train/serve-skew class this placement prevents.

tests\test_config.py:281:def test_config_shallow_section_replacement():

tests\test_config.py:282: """Positive test for Item 2 (config loader): shallow section replacement.

docs\TRACKNET_WSL_SETUP.md:67: "velocity_threshold": 0.02 // normalised shuttle displacement/sec to count as "in flight"

docs\STORAGE_SHARING_MODEL.md:185:13. On-device milestone placement: academy pilot boxes first, consumer app after the WSL-free

docs\archives\HARDENING_LOOP_HANDOFF-COMPLETED-2026-06-14.md:150: - Restored the strong original `test\_get\_returns\_dict\_for\_nested\_sections\_legacy\_compat` (exact name + all legacy dict/nested/scalar/leaf-key/ default/fallback guards that the weaker shim replacement had dropped).

docs\ROADMAP.md:32:pipeline objectively and (2) train an offline replacement for the paid Gemini

backend\ui\assets\index-kJhH4dVE.js:60:[Omitted long matching line]

docs\RALLY_DETECTION_QUALITY_REPORT.md:288:aggressively (so the gate trims **real** rallies instead). The honest, served replacement is **w2**

docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:156:- Shallow section replacement behavior.

docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:361: -

`test\_config\_shallow\_section\_replacement`: verifies loader's shallow **{**defaults, **file_data}** (whole section from file replaces defaults section; no deep merge of sub-keys).

docs\R2_EVAL_METRIC_DESIGN.md:156:- **Default-additive ? ablation-gated replacement:** R2 is reported next to tIoU (augment). The **gate**

docs\R2_EVAL_METRIC_DESIGN.md:188:- **Gate:** AUGMENT now, then ablation-gate the eventual

```

replacement, then deprecate-with-docs ? ADOPTED.**
backend\eval\per_user_regression.py:16:  * **CANDIDATE** = the proposed replacement (a per-user
retrain, or a global baseline upgrade
backend\eval\per_user_regression.py:200:  replacement (a per-user retrain or a baseline
upgrade). Both are scored on the user's own
tests\test_fusion_hybrid.py:152:      # glue (in-court counts -> players_in_court; track
displacement -> mean_player_motion),
docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md:7:**Key outcomes from #166 (merged
c558372):** Hardening loop (T1-T5 + finale #165) COMPLETE and archived. NEXT_STEPS reframed with
explicit P3 start signal (harness LOVO litmus for person features) + full prioritized candidate
list. Gate A wired (opt-in, default-OFF safety per served regression litmus). Loop scheduler
(019ec6d07da5) deleted; no replacement created. Handoff and archived audit updated terminal.
Master also includes #167 (packaging).
docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md:21:[Omitted long matching line]
docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md:45:[Omitted long matching line]
docs\PER_VIDEO_WORKSPACE.md:6:[Omitted long matching line]
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:52:  wholesale replacement. *(Open: the VLM-ceiling
number is open-domain; our amateur footage may differ ? re-measure.)*
backend\pipeline\detectors\fusion_features.py:132:  """Mean per-track centre displacement
between consecutive sampled frames, with x
backend\pipeline\detectors\stub_runner.py:52:      12.0 # per-frame shuttle displacement
during a rally (>> velocity_thresh)
backend\pipeline\detectors\tracknet_runner.py:333:      displacement (normalised by frame
width, per second) exceeds
backend\pipeline\detectors\wasb_infer.py:412:  """Build the ``read_image`` replacement used
during a streaming detector pass.
docs\KHELSUTRA_PRODUCT_OVERVIEW.md:161:- It is **not a replacement for a coach** ? it's a tool
that saves coaches time and adds insight.

```

[Showing results with pagination = limit: 30]

30. user (2026-07-08T05:47:12.246Z)

```

56-from backend.config import (
57-    AppConfig,
58-    StartupCheckError,
59-    enforce_startup_checks,
60-    write_light_default_config, # P2b: batteries-included first-run config seeding
61-)
62-from backend.config import ( # new typed config
--
221-    _maybe_record_job_cost,
222-    _run_claimed_job,
223-    _schedule_next_drain_if_waiting,
224:    configure_drain,
225-    resume_interrupted_remote_jobs,
226-)
227-
--
1238-    the server stays torch-light; the detector healthcheck is the GPU authority. Extracted
so the
1239-    fail-fast wiring + the default-OFF no-op are unit-testable."""
1240-    try:
1241:        enforce_startup_checks(config, logger=logger, include_exposure=True)
1242-    except StartupCheckError as e:
1243:        logger.error("[STARTUP] refusing to start: %s", e)
1244-        sys.exit(1)
--
1632-        # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3): the
1633-        # in-process wake-timer for any parked 'waiting' job died with the old process, so
1634-        # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
1635:        configure_drain(global_config) # drain concurrency from config before the executor
exists (#466)
1636-        _get_executor().submit(_drain_due_jobs, global_config, db_instance)
1637-        # Stamp shared state on the app so route handlers access it via request.app.state

```

1638- # instead of the Optional module-level globals (CODE_CLEANUP_AUDIT P5 / D2).

31. user (2026-07-08T05:47:13.655Z)

```
12:- The model's Literal enum stays in parity with backend/config/presets.PROFILE_PRESETS.
26:  COMPUTE_TARGETS,
27:  PROFILE_PRESETS,
77:# Parity: the model Literal must match the presets table (no drift).
86:  assert set(PROFILE_PRESETS) == set(PROFILES)
96:  assert literal_targets == {"", *COMPUTE_TARGETS}
97:  assert set(COMPUTE_TARGET_FOR_PROFILE.values()) == set(COMPUTE_TARGETS)
270:  """A bad VALUE in config.json hits the typed Literal ? a loud, hard error (same class as
a
```

32. user (2026-07-08T05:47:24.084Z)

```
40:ENV DEBIAN_FRONTEND=noninteractive \
74:ENV MAMBA_ROOT_PREFIX=/opt/conda
115:ENV PYTHONUNBUFFERED=1 \
123:ENTRYPOINT ["python3", "-m", "backend.worker"]
```

33. user (2026-07-08T05:47:28.870Z)

```
108      && python3 -m pip install ".[storage-gcs,cloud-run,auth]"
109
110      # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
111      # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
112      # PYTHONUNBUFFERED: without it the worker's stdout/stderr sit in the block buffer until
process
113      # exit, so Cloud Logging shows a single end-of-run burst instead of live progress
(observed on the
114      # first real M7 runs, 2026-07-03) ? a running job looks stalled and mid-run failures log
nothing.
115      ENV PYTHONUNBUFFERED=1 \
116          WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
117          WASB_REPO_DIR=/opt/models/WASB-SBDT \
118          WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
119          RALLY_DETECTOR_IMPL=native
120
121      # The dispatcher (CloudRunCompute) supplies the per-execution args:
122      # infer --video-uri gs:/// --out-uri gs:/// --done-uri gs:/// --segmenter wasb_hybrid
--set ?
123      ENTRYPOINT ["python3", "-m", "backend.worker"]
```

34. assistant (2026-07-08T05:48:56.824Z)

I have a complete map of the config subsystem. Let me finalize the structured output.

35. user (2026-07-08T05:50:53.606Z)

Structured output provided successfully